

ToricGT: Graph-Token Reasoning with Tropical Ring Attention, Noncommutative Tori, Soft-MoE Routing, and Parameter-Golf Compression

Amelie Schreiber

May 25, 2026

Abstract

This manuscript extends the ToricGT framework into a more formal, didactic, and proof-heavy specification. ToricGT is a graph-to-graph architecture that combines TokenGT-style node and edge tokenization, exact ring evaluation of attention, tropical max-plus heads for dynamic-programming structure, finite noncommutative-torus channels for projective phase memory, and embedding-space Generative Flow Network fine-tuning. We formalize bounded typed graph domains, prove equivariance of tokenization, masking, attention, decoding, and projective toric transports, and reduce bounded graph-to-graph universality to sequence-to-sequence Transformer universality. We then give exact blockwise tropical ring-attention proofs, stability lemmas for active faces, and constructive simulations of Bellman-Ford-style semiring recurrences. The operator-algebra layer is sharpened by normal-form, cocycle, trace, finite Weyl-pair, and irrational-spiral projection theorems. A new Hebrew morphology section defines root-template graphs for biblical and classical Hebrew, including radical alignment, binyan/template structure, source-span graphs, paradigm graphs, and semiring alignment losses. We add a visualization contract for auditing reasoning capacity: equivariance orbits, tropical active faces, toric phase maps, Hebrew paradigm lattices, GFlowNet reward landscapes, and proof-dependency graphs. We integrate Soft-MoE routing as a typed, fully differentiable expert-slot layer for ToricGT graph tokens, prove its permutation equivariance, give low-rank and shared-expert variants for compact artifacts, and specify a Parameter-Golf-constrained micro-ToricGT language-model track with byte-accurate artifact accounting, BPB scoring, quantization-aware training, record-folder checklists, and compliance risk controls. Finally, we integrate PolarQuant-style random pre-conditioning and recursive polar-angle quantization as an optional long-context KV-cache layer for ToricGT ring attention, with perturbation bounds showing when quantized keys and values preserve tropical outputs and active argmax faces. This revision also adds two implementation-facing layers requested for the training program: Soft-MoE feed-forward replacement blocks adapted to graph tokens, and a Parameter-Golf compression track that turns ToricGT ideas into a self-contained micro language model satisfying the OpenAI Model Craft Challenge constraints. The result is a mathematically controlled specification for a small, reproducible model family aimed at graph-structured mathematical, algebraic, tropical, Hebrew-morphological, and parameter-constrained language-model reasoning.

Contents

1	Purpose and high-level architecture	4
2	Typed attributed graph domains	4
2.1	Graph objects	4
3	Tokenization as an equivariant construction	6
3.1	Token maps	6

3.2	Attention and block equivariance	7
4	Universal graph-to-graph approximation	8
5	Tropical ring attention and semiring reasoning	9
5.1	Max-plus heads with provenance	9
5.2	Dynamic-programming gates	11
6	Rotation algebras, noncommutative tori, and spiral projections	12
6.1	Finite Weyl pairs and normal forms	12
6.2	Higher noncommutative tori	13
6.3	Infinite spiral projection	14
6.4	Trace supervision	15
7	Hebrew roots as graph-structured morphology	15
7.1	Why Hebrew morphology belongs in ToricGT	15
7.2	Graph construction algorithm	17
7.3	Hebrew morphology losses	17
7.4	Tropical alignment for root extraction	18
7.5	Hebrew graph tasks	18
8	Reasoning capacity and visualization contract	19
9	Embedding-space GFlowNet fine-tuning	21
10	Interleaving PolarQuant with ToricGT ring attention	22
10.1	Applicability	22
10.2	Recursive polar transform	22
10.3	Polar ring cache	23
10.4	Memory accounting	24
11	Soft-MoE ToricGT blocks	25
11.1	Motivation and placement	25
11.2	Equations	25
11.3	Typed expert bank for ToricGT	27
11.4	Implementation details	27
11.5	Prefix-causal Soft-MoE for language-model scoring	28
11.6	Soft-MoE regularizers and diagnostics	29
12	Parameter-Golf constrained ToricGT	30
12.1	Contest constraints as engineering axioms	30
12.2	Prequential BPB validity	31
12.3	Micro-ToricGT language-model design	32
12.4	Soft-MoE under a 16 MB artifact cap	32
12.5	Distillation from ToricGT to the contest LM	33
12.6	Artifact packing and runtime implementation	33
12.7	Contest record folder checklist	34
12.8	Pragmatic risk controls	34

13 Datasets, synthetic families, and leakage control	34
13.1 Training mixture	34
13.2 Normalized record schema	36
13.3 Graph conversion policy	36
13.4 Leakage-resistant splitting	36
13.5 Output layout and commands	37
13.6 Expected scale and license handling	38
13.7 Synthetic generator specifications	38
14 Model, losses, and training schedule	38
14.1 Architecture	38
14.2 Composite objective	39
14.3 Schedule	39
15 Evaluation and ablations	40
16 Implementation blueprint	40
17 Discussion and limits	41
18 Conclusion	41
A Additional proof details	42
A.1 Hybrid universal approximation	42
A.2 Softmax perturbation with quantized keys	42
B Graph record schema	42
C Recommended hyperparameters	43

1 Purpose and high-level architecture

The original ToricGT proposal joined four pieces: graph-tokenized Transformers, ring attention, tropical max-plus reasoning, and finite noncommutative-torus features. This version makes the claims precise. The paper deliberately separates three levels of assertion.

First, there are exact statements about finite graph-token computations: permutation equivariance, ring-schedule exactness, projective phase consistency, and stability of max-plus argmax faces. These can be proved without empirical assumptions. Second, there are compact-domain approximation statements. These are universal-approximation results over bounded graph families, not claims about all finite graphs of arbitrary size. Third, there are experimental design claims: if one trains on finite shadows of rotation algebras, braid groups, Coxeter groups, Hebrew morphological paradigms, and tropical graph algorithms, the correct evaluations are held-out families and graph-similarity-controlled splits, not random example-level splits.

Model family. A ToricGT instance consists of

$$\text{ToricGT} = D \circ \Phi_L \circ \cdots \circ \Phi_1 \circ T, \quad (1.1)$$

where T is a graph-tokenizer, each Φ_ℓ is a token-permutation-equivariant Transformer block with softmax, tropical, or hybrid attention, and D is a shared node/edge/graph decoder. Optional side modules supply toric phase transports, a GFlowNet policy head, and a PolarQuant cache for long-context inference.

Core design invariants. The model is engineered around five invariants.

1. *Graph-to-graph equivariance*: relabeling a graph relabels node and edge outputs in the same way.
2. *Exact schedule/function separation*: ring attention changes the evaluation schedule, not the attention function being evaluated.
3. *Semiring visibility*: tropical heads expose active maximizers and margins, so dynamic-programming decisions can be visualized and tested.
4. *Projective algebraic memory*: toric phase channels encode twisted commutation and cocycles without pretending that a finite neural network faithfully represents an infinite C^* -algebra.
5. *Auditable reasoning*: every run emits diagnostics that make equivariance, active faces, phase consistency, root-template alignments, and GFlowNet diversity inspectable.

2 Typed attributed graph domains

2.1 Graph objects

Definition 2.1 (Typed attributed directed multigraph). A typed attributed graph is a tuple

$$G = (V, E, s, t, \tau_V, \tau_E, x, e, m_V, m_E), \quad (2.1)$$

where $V = \{1, \dots, n\}$ is a finite vertex set, $E = \{1, \dots, m\}$ is a finite edge-token set, $s, t : E \rightarrow V$ are source and target maps, $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are finite type maps, $x_i \in \mathbb{R}^{d_v}$ and $e_a \in \mathbb{R}^{d_e}$ are bounded real attributes, and m_V, m_E are masks when the graph is embedded into a padded domain.

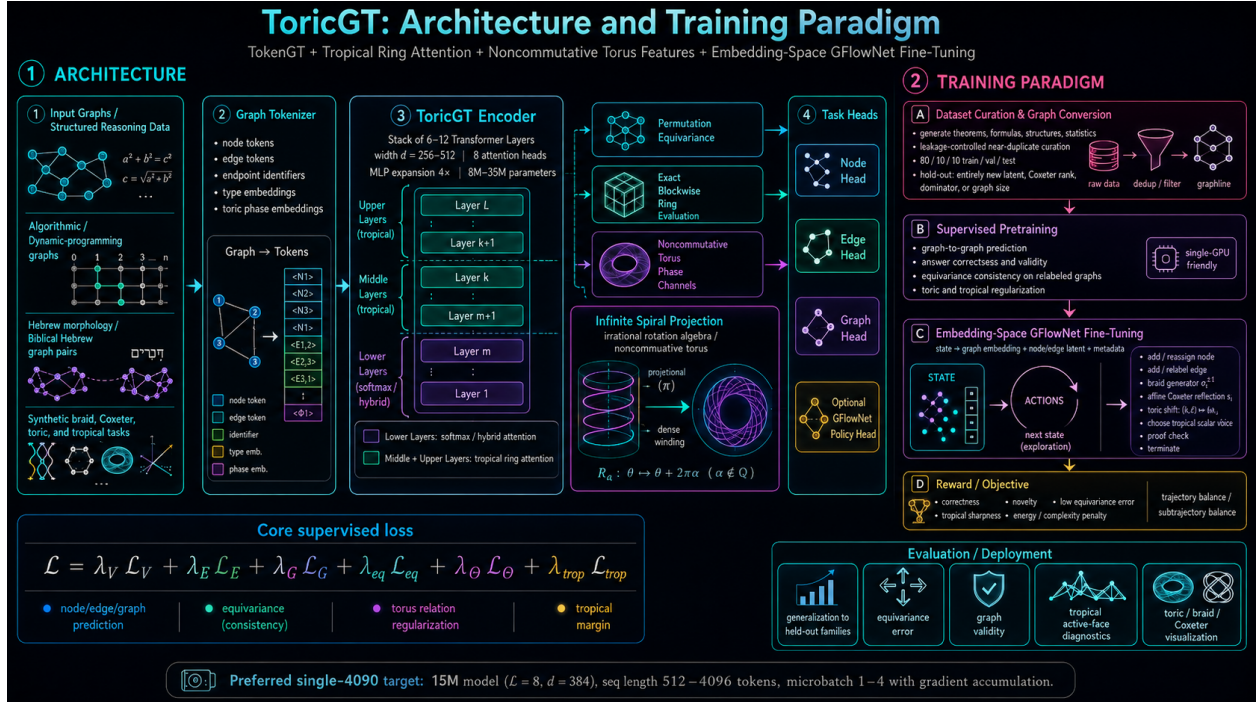


Figure 1: Dark-mode architecture and training-paradigm diagram. The central encoder mixes lower softmax/hybrid layers with middle and upper tropical ring-attention layers, while graph tokens carry endpoint, type, and toric phase features. Training combines supervised graph-to-graph losses with embedding-space GFlowNet exploration.

The edge set is treated as a set of edge tokens rather than as raw file-order rows. This matters: file order is rarely a graph invariant. For a node relabeling $\pi \in S_n$, an edge a with endpoints $(s(a), t(a))$ is transformed to an edge with endpoints $(\pi s(a), \pi t(a))$. If edge tokens are stored by a canonical endpoint order, this induces an edge permutation ρ_π ; if edge tokens are stored unordered, ρ_π is any consistent action on the edge-token set.

Definition 2.2 (Padded bounded graph cell). Fix $N_{\max}$ and $M_{\max}$. A padded graph cell $\mathcal{G}_{n,m,I,\tau}$ consists of graphs with exactly $n \leq N_{\max}$ valid nodes, $m \leq M_{\max}$ valid edges, fixed endpoint pattern $I \in \{1, \dots, n\}^{m \times 2}$, fixed node and edge type pattern τ , and attributes in compact sets $K_V \subset \mathbb{R}^{d_v}$ and $K_E \subset \mathbb{R}^{d_e}$. The full bounded domain is the finite union

$$\mathcal{G}_{\leq N, \leq M} = \bigcup_{n \leq N, m \leq M, I, \tau} \mathcal{G}_{n,m,I,\tau}. \quad (2.2)$$

Lemma 2.3 (Compactness of cells). *Every graph cell $\mathcal{G}_{n,m,I,\tau}$ is compact. The bounded padded graph space $\mathcal{G}_{\leq N, \leq M}$ is a finite union of compact cells and is therefore compact when equipped with the disjoint-union topology or with a mask topology that separates cells.*

Proof. Fix n, m, I, τ . The only continuous degrees of freedom are node attributes $X \in K_V^n$ and edge attributes $E \in K_E^m$. Finite products of compact sets are compact by Tychonoff's theorem in the finite case, equivalently by Heine-Borel when the sets are closed and bounded subsets of Euclidean space. Endpoint and type data are fixed discrete parameters in the cell. Hence $\mathcal{G}_{n,m,I,\tau} \simeq K_V^n \times K_E^m$ is compact. There are finitely many masks, endpoint patterns, and type patterns under fixed N and M , so the union is finite. A finite union of compact sets is compact. $\square$

Lemma 2.4 (Induced edge permutations form an action). *Suppose edge canonicalization is label-compatible: for all $\pi, \sigma \in S_n$, canonicalizing after σ and then after π is the same as canonicalizing after $\pi\sigma$. Then $\rho_{\pi\sigma} = \rho_\pi \rho_\sigma$ and $\rho_{\text{id}} = \text{id}$.*

Proof. Let a be an edge token with endpoints (u, v) . Applying σ sends the endpoints to $(\sigma u, \sigma v)$ and moves the token to position $\rho_\sigma(a)$. Applying π next sends endpoints to $(\pi\sigma u, \pi\sigma v)$ and token position to $\rho_\pi(\rho_\sigma(a))$. Direct application of $\pi\sigma$ gives the same endpoint pair. By label-compatible canonicalization the same edge-token position is assigned, so $\rho_{\pi\sigma}(a) = \rho_\pi \rho_\sigma(a)$ for every a . The identity relabeling leaves endpoints and canonical order unchanged. $\square$

Definition 2.5 (Graph-to-graph equivariance). Let $P_\pi = \pi \oplus \rho_\pi$ be the induced token permutation on node and edge outputs. A graph-to-graph map $F : \mathcal{G} \rightarrow \mathcal{Y}_V^n \times \mathcal{Y}_E^m \times \mathcal{Y}_G$ is equivariant if

$$F(\pi \cdot G) = (P_\pi F_{V,E}(G), F_G(G)) \quad (2.3)$$

for every admissible relabeling π . Its graph-level component is invariant when $F_G(\pi \cdot G) = F_G(G)$.

3 Tokenization as an equivariant construction

3.1 Token maps

A graph tokenizer maps typed attributed graphs to token matrices. Node and edge tokens have distinct type channels, endpoint channels, structural identifiers, and optional toric phase channels. For a node i and an edge token $a = (u, v)$,

$$t_i^V = \phi_V(x_i, \tau_V(i)) + \eta_i + \kappa_i + \theta_i, \quad (3.1)$$

$$t_a^E = \phi_E(e_a, \tau_E(a)) + \psi(\eta_u, \eta_v) + \zeta_a + \kappa_a + \theta_a. \quad (3.2)$$

Here η_i is an equivariant node identifier, ζ_a is an edge identifier, κ denotes optional structural features such as degree or Laplacian information, and θ denotes optional toric phase features. The maps ϕ_V, ϕ_E, ψ are shared across all nodes and edges.

Assumption 3.1 (Equivariant identifiers). The identifier fields obey

$$\eta_i(\pi G) = \eta_{\pi^{-1}i}(G), \quad \zeta_a(\pi G) = \zeta_{\rho_\pi^{-1}a}(G), \quad (3.3)$$

and structural/phase channels obey the analogous transformation law.

Lemma 3.2 (Tokenization equivariance). *Under equivariant identifiers and shared endpoint maps,*

$$T(\pi \cdot G) = P_\pi T(G). \quad (3.4)$$

Proof. For a node token,

$$\begin{aligned} t_i^V(\pi G) &= \phi_V(x_{\pi^{-1}i}, \tau_V(\pi^{-1}i)) + \eta_{\pi^{-1}i} + \kappa_{\pi^{-1}i} + \theta_{\pi^{-1}i} \\ &= (P_\pi T(G))_i. \end{aligned} \quad (3.5)$$

For an edge token a in the relabeled graph, its preimage under ρ_π is $\rho_\pi^{-1}a$. Its source and target identifiers are the permuted identifiers of the original endpoints. Since ϕ_E and ψ are shared, the edge token after relabeling equals the edge token at $\rho_\pi^{-1}a$ before relabeling. Concatenating node and edge rows gives $P_\pi T(G)$. $\square$

Lemma 3.3 (Mask equivariance). *Let $M(G) \in \{0, -\infty\}^{L \times L}$ be an additive attention mask whose entries depend only on token validity, token types, and graph incidence relations. Then*

$$M(\pi G) = P_\pi M(G) P_\pi^\top. \quad (3.7)$$

Proof. Validity masks permute with valid token rows. Type masks are preserved because node tokens map to node tokens and edge tokens map to edge tokens. Incidence masks are preserved because a is incident to i exactly when $\rho_\pi a$ is incident to πi . Thus every masked or unmasked pair is carried to its relabeled pair, which is exactly conjugation by P_π . $\square$

3.2 Attention and block equivariance

For token matrix $Z \in \mathbb{R}^{L \times d}$, define

$$Q = ZW_Q, \text{quad}K = ZW_K, \quad V = ZW_V, \quad S = QK^\top / \sqrt{d_h}. \quad (3.8)$$

A softmax head computes $\text{softmax}(S + M)V$. A max-plus tropical head computes

$$Y_{ic} = \max_j \{S_{ij} + M_{ij} + V_{jc}\}. \quad (3.9)$$

Lemma 3.4 (Softmax and tropical attention are token equivariant). *For any token permutation matrix P and mask satisfying $M(PZ) = PM(Z)P^\top$,*

$$\text{Attn}(PZ) = P \text{Attn}(Z) \quad (3.10)$$

for softmax attention and for tropical attention.

Proof. Linear projections commute with row permutations: $Q(PZ) = PQ(Z)$ and similarly for K, V . Hence the score matrix becomes

$$S(PZ) = PQK^\top P^\top / \sqrt{d_h} = PS(Z)P^\top. \quad (3.11)$$

For softmax, row-wise exponentiation and normalization are invariant under simultaneous row and column reindexing:

$$\text{softmax}(PSP^\top + PMP^\top) = P \text{softmax}(S + M)P^\top. \quad (3.12)$$

Multiplication by PV then yields $P \text{softmax}(S + M)V$. For tropical attention, fix output row i and coordinate c . If p is the permutation represented by P , then

$$Y'_{ic} = \max_j \{(PSP^\top + PMP^\top)_{ij} + (PV)_{jc}\} \quad (3.13)$$

$$= \max_j \{S_{p^{-1}i, p^{-1}j} + M_{p^{-1}i, p^{-1}j} + V_{p^{-1}j, c}\} \quad (3.14)$$

$$= \max_{j'} \{S_{p^{-1}i, j'} + M_{p^{-1}i, j'} + V_{j'c}\} = Y_{p^{-1}i, c}. \quad (3.15)$$

Thus $Y' = PY$. $\square$

Lemma 3.5 (Row-wise sublayers are equivariant). *Shared feed-forward networks, residual connections, dropout with shared distribution over rows, and layer normalization applied row-wise commute with token permutations.*

Proof. A shared feed-forward network f acts as $(f(Z_1), \dots, f(Z_L))$. Permuting rows before or after applying the same f gives the same sequence of rows. Residual addition is row-wise vector addition. Layer normalization uses statistics within a row, not across row labels. Dropout is exactly equivariant when the dropout mask is permuted with rows, and distributionally equivariant when masks are independently sampled from the same row-wise distribution. $\square$

Theorem 3.6 (TokenGT stack equivariance). *A ToricGT encoder built from equivariant tokenization, equivariant masks, shared attention projections, shared row-wise feed-forward layers, residuals, layer normalization, and shared node/edge decoders is graph-to-graph equivariant.*

Proof. Tokenization maps G to Z and πG to $P_\pi Z$. Each encoder block maps $P_\pi Z$ to $P_\pi \Phi(Z)$ by the previous two lemmas. A composition of equivariant maps is equivariant by induction on layers. Shared node decoders apply the same map to every node-token row, and shared edge decoders apply the same map to every edge-token row. Therefore decoding also commutes with P_π . Graph-level pooling by sum, mean, log-sum-exp, max, or attention with a permutation-invariant query is invariant because it is a symmetric function of token rows. $\square$

4 Universal graph-to-graph approximation

Assumption 4.1 (Token separability). For each fixed graph cell $\mathcal{G}_{n,m,I,\tau}$, the tokenizer T is continuous and injective. Since the domain is compact and the codomain is Hausdorff, T is a homeomorphism from $\mathcal{G}_{n,m,I,\tau}$ onto $T(\mathcal{G}_{n,m,I,\tau})$.

Theorem 4.2 (Fixed-cell graph-to-graph universality). *Let $F : \mathcal{G}_{n,m,I,\tau} \rightarrow \mathbb{R}^{(n+m) \times d_o}$ be continuous and equivariant under all relabelings preserving the cell. Under token separability, for every $\varepsilon > 0$ there exists a TokenGT-style Transformer Φ without absolute position embeddings such that*

$$\sup_{G \in \mathcal{G}_{n,m,I,\tau}} \|\Phi(T(G)) - F(G)\| < \varepsilon, \quad (4.1)$$

and Φ is token-permutation equivariant.

Proof. Let $K_T = T(\mathcal{G}_{n,m,I,\tau})$. The set K_T is compact because T is continuous and the graph cell is compact. Since T is a homeomorphism onto K_T , the conjugate map

$$\tilde{F} = F \circ T^{-1} : K_T \rightarrow \mathbb{R}^{(n+m) \times d_o} \quad (4.2)$$

is continuous. Equivariance of F and T gives, for every admissible P_π ,

$$\tilde{F}(P_\pi Z) = F(T^{-1}(P_\pi Z)) = F(\pi T^{-1}(Z)) = P_\pi F(T^{-1}(Z)) = P_\pi \tilde{F}(Z). \quad (4.3)$$

Yun et al.’s sequence-to-sequence Transformer universality theorem gives an equivariant Transformer approximating any continuous equivariant map on a compact token domain. Apply that theorem to $\tilde{F}$ on K_T . The approximation inequality follows after composing with T . $\square$

Corollary 4.3 (Bounded variable-size universality). *Let $\mathcal{G}_{\leq N, \leq M}$ be a finite union of graph cells. If F is continuous on each cell, equivariant, and compatible with padding masks, then a masked TokenGT model uniformly approximates F on the bounded padded domain.*

Proof. Apply the fixed-cell theorem to every cell. There are finitely many cells. The model can read mask and type indicators; shared feed-forward layers can approximate a continuous partition-of-unity-like selector over this finite discrete partition. Since the maximum of finitely many approximation errors can be made below ε , a single masked model approximates all cells. Mask equivariance preserves the group action. $\square$

Remark 4.4 (Scope). The theorem is not a claim that one finite model uniformly solves all graph sizes, all denominators q , all braid lengths, all Hebrew lexemes, or the entire irrational rotation algebra. It is a compact-domain theorem. Infinite objects enter the experimental program through finite presentations, finite approximants, finite graphs, and curriculum generalization tests.

5 Tropical ring attention and semiring reasoning

5.1 Max-plus heads with provenance

Let $\mathbb{T}_{\max} = \mathbb{R} \cup \{-\infty\}$ with $a \oplus b = \max(a, b)$ and $a \otimes b = a + b$. A tropical attention head computes

$$Y_{ic} = \bigoplus_{j=1}^L S_{ij} \otimes V_{jc} = \max_j (S_{ij} + V_{jc}). \quad (5.1)$$

Its provenance set is

$$A_{ic} = \arg \max_j (S_{ij} + V_{jc}). \quad (5.2)$$

When A_{ic} is a singleton, the output is locally affine. The top-two tropical margin is

$$\Delta_{ic} = z_{ic}^{(1)} - z_{ic}^{(2)}, \quad z_{icj} = S_{ij} + V_{jc}, \quad (5.3)$$

where $z^{(1)}$ and $z^{(2)}$ are the largest and second-largest candidate values.

Theorem 5.1 (Exact blockwise tropical ring attention). *Partition the key-value indices into blocks $B_1, \dots, B_R$. Let*

$$\hat{Y}_{ic}^{(r)} = \max_{j \in B_r} (S_{ij} + V_{jc}), \quad \hat{Y}_{ic} = \max_r \hat{Y}_{ic}^{(r)}. \quad (5.4)$$

Then $\hat{Y}_{ic} = Y_{ic}$ for every query i and coordinate c , independent of block order.

Proof. The blocks form a partition of $\{1, \dots, L\}$. Thus

$$Y_{ic} = \max_{j \in \cup_r B_r} (S_{ij} + V_{jc}) \quad (5.5)$$

$$= \max \left(\max_{j \in B_1} (S_{ij} + V_{jc}), \dots, \max_{j \in B_R} (S_{ij} + V_{jc}) \right) \quad (5.6)$$

$$= \max_r \hat{Y}_{ic}^{(r)}. \quad (5.7)$$

The binary maximum is associative and commutative, so the order in which blocks arrive around the ring cannot change the exact real-arithmetic result. Floating-point implementations differ only by standard numerical roundoff and tie-breaking behavior. $\square$

Corollary 5.2 (Exact provenance under ring evaluation). *If each block returns both its block maximum and the local argmax set, then a second maximum over block maxima returns the same global provenance set A_{ic} as full tropical attention.*

Proof. Let $m_r = \max_{j \in B_r} z_j$ and $m = \max_r m_r$. A candidate j is globally active exactly when $z_j = m$. This holds exactly when j is locally active in a block B_r whose block value $m_r = m$. Taking the union of local argmax sets over globally maximal blocks gives precisely the global argmax set. $\square$

Lemma 5.3 (Lipschitz stability of tropical attention). *Let S, V and $\hat{S}, \hat{V}$ satisfy*

$$\|S - \hat{S}\|_\infty \leq \epsilon_S, \quad \|V - \hat{V}\|_\infty \leq \epsilon_V. \quad (5.8)$$

Then the tropical outputs satisfy

$$\|Y - \hat{Y}\|_\infty \leq \epsilon_S + \epsilon_V. \quad (5.9)$$

If the original margin $\Delta_{ic} > 2(\epsilon_S + \epsilon_V)$, then the active argmax index for (i, c) is unchanged under perturbation.

Proof. For every candidate j ,

$$\left| (S_{ij} + V_{jc}) - (\hat{S}_{ij} + \hat{V}_{jc}) \right| \leq \epsilon_S + \epsilon_V. \quad (5.10)$$

The maximum function is 1-Lipschitz in ℓ_∞ :

$$\left| \max_j a_j - \max_j b_j \right| \leq \max_j |a_j - b_j|. \quad (5.11)$$

This proves the output bound. For argmax stability, let j^* be the unique original maximizer and let $j \neq j^*$. After perturbation,

$$\hat{z}_{j^*} - \hat{z}_j \geq (z_{j^*} - (\epsilon_S + \epsilon_V)) - (z_j + (\epsilon_S + \epsilon_V)) \quad (5.12)$$

$$\geq \Delta_{ic} - 2(\epsilon_S + \epsilon_V) > 0. \quad (5.13)$$

Thus j^* remains the unique maximizer. $\square$

5.2 Dynamic-programming gates

Definition 5.4 (Equivariant max-plus graph circuit). An equivariant max-plus graph circuit is a finite directed acyclic circuit whose inputs are graph-token features, whose gates are shared over token orbits, and whose primitive gates compute

$$g(x) = \max_{r \in [R]} (a_r^\top x + b_r) \quad (5.14)$$

or ordinary affine maps, with outputs transforming equivariantly under graph relabeling.

Lemma 5.5 (Tropical heads implement max-plus affine gates). *For a bounded token domain, a tropical attention head followed by shared affine maps can implement, or uniformly approximate, a finite max-plus affine gate $g(x) = \max_r (a_r^\top x + b_r)$.*

Proof. Create one candidate key-value channel for each affine form. A shared linear map computes the candidate score $a_r^\top x + b_r$ and stores it as $S_{ir} + V_{rc}$ for an appropriate query row i and output coordinate c . Tropical attention takes the maximum over r . If exact candidate construction is not available because candidate identities are embedded continuously, a feed-forward network approximates the candidate scores uniformly on the compact domain, and the Lipschitz property of maximum transfers this approximation to the gate output. $\square$

Proposition 5.6 (One Bellman-Ford relaxation as tropical attention). *For a directed weighted graph with current distances D_u and edge weights $w_{u \rightarrow v}$, the relaxation*

$$D'_v = \min \left(D_v, \min_{u: u \rightarrow v} (D_u + w_{u \rightarrow v}) \right) \quad (5.15)$$

can be implemented by a min-plus variant of tropical attention with an incidence mask.

Proof. Use the identity $\min_j z_j = -\max_j (-z_j)$. For each query node v , mask keys to include v itself and predecessors u satisfying $u \rightarrow v$. Give the self-candidate value D_v and predecessor candidate value $D_u + w_{u \rightarrow v}$. Applying negative max-plus attention computes the minimum over these candidates. The incidence mask is equivariant by mask equivariance. Therefore the relaxation is equivariant. $\square$

Theorem 5.7 (Tropical TokenGT approximation of bounded semiring algorithms). *Let A be a graph algorithm over bounded graphs that can be unrolled into T finite max-plus, min-plus, or Boolean semiring circuit layers with shared local rules and equivariant masks. Then a ToricGT stack with tropical heads and shared feed-forward layers uniformly approximates the unrolled algorithm on the bounded graph domain and is graph equivariant.*

Proof. Each primitive semiring gate is either a max/min over finitely many affine candidates, an addition of finite channels, or a Boolean threshold. Max/min gates are implemented by tropical heads by the previous lemma; additions are affine maps; thresholds can be uniformly approximated away from zero-margin decision boundaries by ReLU feed-forward layers, or represented exactly if Boolean values are embedded as 0 and $-\infty$ with exact masks. Composing the approximating layers gives an approximation to the unrolled circuit. Equivariance follows because every gate is shared over node and edge orbits and every mask transforms by conjugation. $\square$

Tropical active path and max-plus envelope

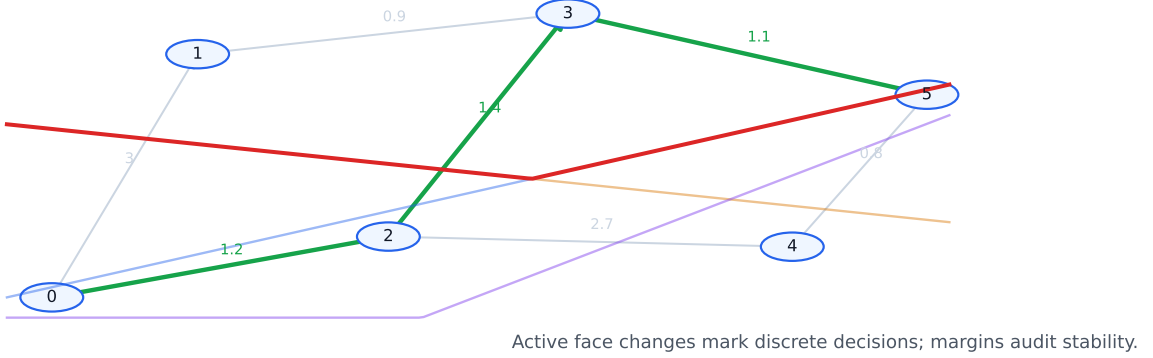


Figure 2: Tropical reasoning diagnostics. Left: a shortest-path-style active predecessor path. Right: a piecewise-linear tropical envelope whose active face and margin provide a direct explanation of a discrete decision.

6 Rotation algebras, noncommutative tori, and spiral projections

6.1 Finite Weyl pairs and normal forms

For $\theta \in \mathbb{R}$, the rotation algebra $\mathcal{A}_\theta$ is generated by unitaries U, V satisfying

$$VU = e^{2\pi i \theta} UV. \quad (6.1)$$

For rational $\theta = p/q$, finite clock and shift matrices give an exact Weyl pair. Let $\omega = e^{2\pi i p/q}$,

$$C_q = \text{diag}(1, \omega, \omega^2, \dots, \omega^{q-1}), \quad S_q e_j = e_{j-1 \bmod q}. \quad (6.2)$$

Lemma 6.1 (Finite clock-shift relation). *The matrices S_q and C_q satisfy*

$$S_q C_q = \omega C_q S_q. \quad (6.3)$$

Proof. Apply both sides to a basis vector e_j , with indices interpreted modulo q . Since $C_q e_j = \omega^j e_j$ and $S_q e_j = e_{j-1}$,

$$S_q C_q e_j = \omega^j e_{j-1}. \quad (6.4)$$

On the other hand,

$$\omega C_q S_q e_j = \omega C_q e_{j-1} = \omega \cdot \omega^{j-1} e_{j-1} = \omega^j e_{j-1}. \quad (6.5)$$

The two operators agree on every basis vector, so $S_q C_q = \omega C_q S_q$. $\square$

Remark 6.2 (Convention discipline). The sign of the clock-shift phase is a common source of synthetic-data bugs. Every generated record must store the shift convention, the normal-form order, and whether the relation is $VU = \omega UV$ or $UV = \omega VU$.

Lemma 6.3 (Rotation-word normal form). *Let $VU = \omega UV$, where $\omega = e^{2\pi i \theta}$. Every word in $U^{\pm 1}, V^{\pm 1}$ reduces to a unique expression*

$$\omega^c U^a V^b, \quad (6.6)$$

where $a, b, c \in \mathbb{Z}$, after fixing the normal-form order U before V . If the word is $W = X_1 \cdots X_T$ and $\deg(X_t) = (u_t, v_t) \in \mathbb{Z}^2$, then

$$a = \sum_{t=1}^T u_t, \quad b = \sum_{t=1}^T v_t, \quad c = \sum_{1 \leq r < s \leq T} v_r u_s, \quad (6.7)$$

with signs interpreted by the same commutation convention.

Proof. The relation $VU = \omega UV$ allows any adjacent out-of-order pair VU to be swapped into UV at the cost of multiplying by ω . Similarly, inverse pairs cancel using $UU^{-1} = U^{-1}U = VV^{-1} = V^{-1}V = 1$, and inverse commutations follow by multiplying the basic relation by inverses. Repeated adjacent swaps bubble all U powers to the left and all V powers to the right. The net exponents a and b are additive because U and V powers combine within their own generators. The phase exponent counts signed crossings in which a V contribution originally appears before a later U contribution; this is exactly $\sum_{r < s} v_r u_s$. Confluence follows because this crossing count depends only on the ordered degree sequence, and all swap sequences that sort the word have the same inversion count in the abelianized degree lattice with the same bilinear cocycle. $\square$

6.2 Higher noncommutative tori

A higher noncommutative torus $\mathcal{A}_\Theta$ is generated by $U_1, \dots, U_n$ with

$$U_j U_k = e^{2\pi i \Theta_{jk}} U_k U_j, \quad (6.8)$$

where $\Theta \in \mathbb{R}^{n \times n}$ is skew-symmetric. For $a, b \in \mathbb{Z}^n$, define the cocycle

$$\sigma_\Theta(a, b) = \exp(2\pi i a^\top \Theta b). \quad (6.9)$$

Lemma 6.4 (Bicharacter identities). *For $a, b, c \in \mathbb{Z}^n$,*

$$\sigma_\Theta(a + b, c) = \sigma_\Theta(a, c) \sigma_\Theta(b, c), \quad (6.10)$$

$$\sigma_\Theta(a, b + c) = \sigma_\Theta(a, b) \sigma_\Theta(a, c), \quad (6.11)$$

$$\sigma_\Theta(a, b) \sigma_\Theta(b, a) = 1. \quad (6.12)$$

Proof. The first two identities follow from bilinearity:

$$(a + b)^\top \Theta c = a^\top \Theta c + b^\top \Theta c, \quad (6.13)$$

and similarly in the second argument. Exponentiating turns sums into products. For the third identity, skew-symmetry gives $b^\top \Theta a = -a^\top \Theta b$. Hence the exponents sum to zero. $\square$

Proposition 6.5 (Projective equivariance of toric transports). *Let T_a be a lattice transport on toric channels satisfying*

$$T_a T_b = \sigma_\Theta(a, b) T_{a+b}. \quad (6.14)$$

If token relabelings act on rows and T_a acts only on toric feature channels attached equivariantly to each row, then ToricGT layers built from shared maps, attention, and T_a are graph-equivariant and projectively lattice-equivariant.

Proof. A graph relabeling P_π acts by row permutation. A toric transport T_a acts within the channel vector of every row by the same linear operator or by an endpoint-derived equivariant channel rule. Therefore $P_\pi T_a = T_a P_\pi$. The only noncommutativity among transports is the scalar cocycle $\sigma_\Theta(a, b)$, which yields projective equivariance. Since all other sublayers are row-shared and equivariant, the composition retains graph equivariance and the stated projective lattice law. $\square$

Spiral diagnostics for irrational rotations

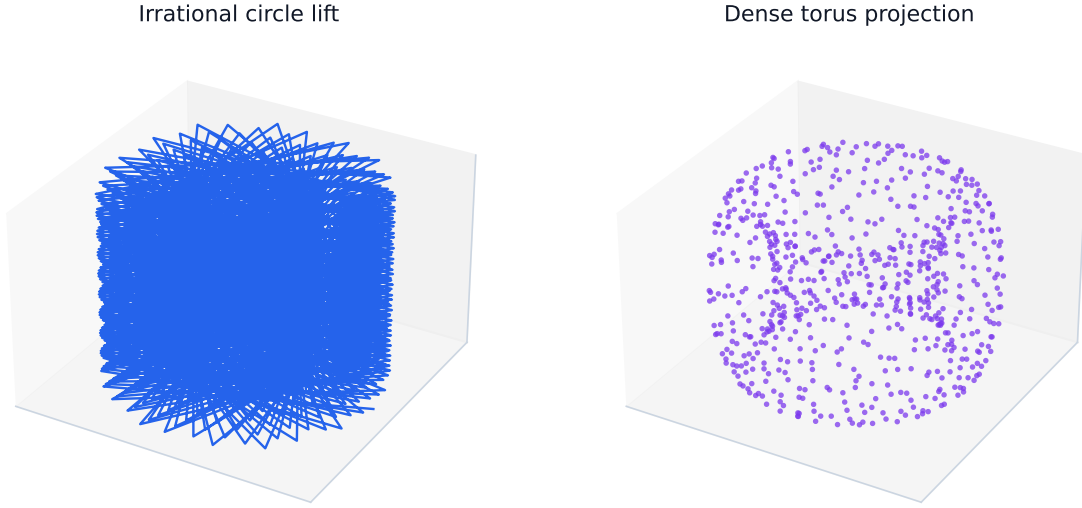


Figure 3: Infinite spiral-projection diagnostic for the rotation algebra and noncommutative torus. The cylinder shows irrational rotation by α ; the torus projection visualizes dense phase winding and finite phase-channel truncations.

6.3 Infinite spiral projection

The infinite spiral view is a visualization, not a finite representation theorem. It makes the irrational rotation visible: repeated additions of α never close when α is irrational, and the orbit winds densely.

Lemma 6.6 (Density of an irrational circle orbit). *If $\alpha \notin \mathbb{Q}$, then the set $\{k\alpha \bmod 1 : k \in \mathbb{N}\}$ is dense in $[0, 1)$.*

Proof. Fix $N \in \mathbb{N}$. Consider the $N + 1$ points $0, \alpha, 2\alpha, \dots, N\alpha$ modulo 1. Partition $[0, 1)$ into N intervals of length $1/N$. By the pigeonhole principle, two points $r\alpha$ and $s\alpha$ lie in the same interval, so $\delta = (s - r)\alpha \bmod 1$ has distance at most $1/N$ from 0. Since α is irrational, $\delta \neq 0$. The arithmetic progression $0, \delta, 2\delta, \dots$ modulo 1 visits every interval of length roughly $1/N$ before wrapping. Because N was arbitrary, every open interval contains some $k\alpha \bmod 1$. $\square$

Definition 6.7 (Spiral projection feature). For irrational α and truncation length K , define the spiral-projection feature path

$$\Gamma_K(k) = (\cos(2\pi k\alpha), \sin(2\pi k\alpha), k/K), \quad 0 \leq k < K, \quad (6.15)$$

and the projected torus path

$$\Pi_K(k) = ((R + r \cos(2\pi k\beta)) \cos(2\pi k\alpha), (R + r \cos(2\pi k\beta)) \sin(2\pi k\alpha), r \sin(2\pi k\beta)), \quad (6.16)$$

where $1, \alpha, \beta$ are rationally independent.

Proposition 6.8 (Finite feature convergence of phases). *Let p_k/q_k be continued-fraction approximants to α . Then*

$$\left| e^{2\pi i p_k/q_k} - e^{2\pi i \alpha} \right| \leq 2\pi |p_k/q_k - \alpha| \rightarrow 0. \quad (6.17)$$

Proof. For real x, y , $|e^{ix} - e^{iy}| = 2|\sin((x - y)/2)| \leq |x - y|$. Set $x = 2\pi p_k/q_k$ and $y = 2\pi\alpha$. Continued-fraction convergence gives $p_k/q_k \rightarrow \alpha$. $\square$

6.4 Trace supervision

The canonical trace on the rotation algebra satisfies

$$\tau(U^a V^b) = \begin{cases} 1, & a = b = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6.18)$$

For a finite Weyl-pair approximation, the normalized trace $q^{-1} \text{Tr}(C_q^a S_q^b)$ obeys the same selection rule modulo q .

Lemma 6.9 (Finite trace selection). *For the q -dimensional Weyl pair, if $a, b \in \mathbb{Z}$, then*

$$\frac{1}{q} \text{Tr}(C_q^a S_q^b) = 0 \quad (6.19)$$

unless $a \equiv b \equiv 0 \pmod{q}$, in which case the normalized trace is 1.

Proof. If $b \not\equiv 0 \pmod{q}$, then S_q^b has no fixed basis vector, so $C_q^a S_q^b$ has zero diagonal and zero trace. If $b \equiv 0 \pmod{q}$, then $S_q^b = I$ and

$$\text{Tr}(C_q^a) = \sum_{j=0}^{q-1} \omega^{aj}. \quad (6.20)$$

This geometric sum is q when $\omega^a = 1$, equivalently $a \equiv 0 \pmod{q}$, and is 0 otherwise. $\square$

7 Hebrew roots as graph-structured morphology

7.1 Why Hebrew morphology belongs in ToricGT

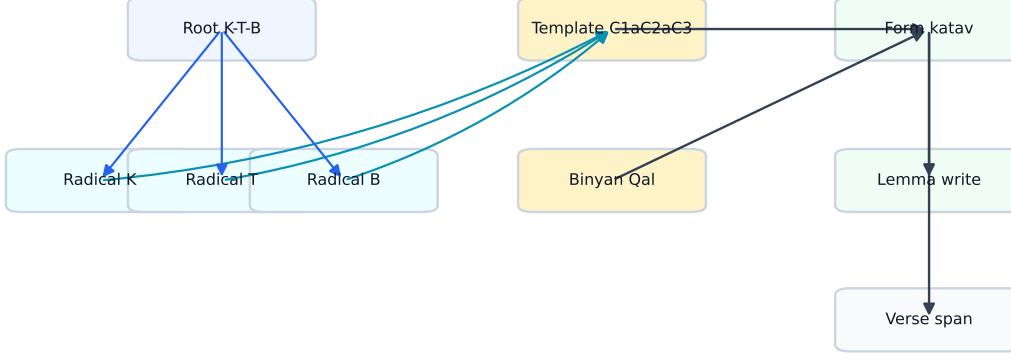
Biblical and classical Hebrew morphology is naturally graph-structured. A surface form is not merely a string: it is related to a lexical root, ordered radicals, a vocalic/consonantal template, a binyan or stem, prefixes and suffixes, morphosyntactic features, a lemma, a gloss, and a source span. These objects form a typed graph with many-to-many dependencies. A root such as כתב (transliterated K-T-B) participates in forms such as *katav* (כָּתַב, “he wrote”), *kotev* (writing/writer), and *miktav* (letter or writing). A model trained only on linear strings must rediscover this structured factorization; a graph-token model can represent it explicitly.

Classical grammars describe Hebrew and other Semitic languages through roots, stems, and patterns; computational morphology often implements such analyses with finite-state or two-level systems [19, 20, 21, 22, 23, 24]. ToricGT does not replace a morphological analyzer. It consumes analyzer outputs, text spans, and uncertain analyses as typed graph evidence, and learns graph-to-graph tasks over those analyses.

Definition 7.1 (Hebrew root-template record). A Hebrew morphology record is a typed graph

$$G_H = (V_H, E_H, \tau, \rho, x, \ell, y) \quad (7.1)$$

Root-template morphology as a typed graph



Records preserve analyzer uncertainty, paradigm edges, source spans, and semiring alignment losses.

Figure 4: A root-template graph for K-T-B. The actual graph record can also store Hebrew glyphs and source spans; the figure uses transliteration for directional readability.

with node types

$$\mathcal{T}_V^H = \{\text{Root, Radical, Pat, Binyan, Feat, Prefix, Suffix, SurfaceForm, Lemma, Gloss, VerseSpan, Paradigm, AnalyzerHypothesis}\}, \quad (7.2)$$

and edge types

$$\mathcal{T}_E^H = \{\text{has_radical, fills_slot, has_template, has_binyan, has_feature, attested_at, glosses, same_root, paradigm_member, analyzer_score}\}. \quad (7.3)$$

The target y may include root identity, radical-slot alignment, binyan, feature bundle, lemma, gloss, paradigm membership, and confidence.

Definition 7.2 (Root-template rendering relation). Let $r = (r_1, r_2, r_3)$ be an ordered radical tuple, p a template with consonant slots C_1, C_2, C_3 and vocalic/affix material, b a binyan, and f a feature bundle. The rendering relation is a partial relation

$$\mathcal{R}_H \subseteq \text{Root} \times \text{Pat} \times \text{Binyan} \times \text{Feat} \times \text{SurfaceForm} \quad (7.4)$$

where $(r, p, b, f, w) \in \mathcal{R}_H$ if w is licensed by inserting radicals into template slots and applying the orthographic/phonological rules attached to b and f .

Remark 7.3 (Root ambiguity). A Hebrew surface string can have multiple analyses. Matres lectionis, missing niqqud, weak roots, gutturals, homographs, and clitic morphology can all create ambiguity. The graph schema therefore allows multiple analyzer-hypothesis nodes with scores, instead of forcing a single gold analysis when the evidence is uncertain.

7.2 Graph construction algorithm

Given a tokenized Hebrew or biblical-Hebrew text with optional morphology, construct G_H as follows.

1. Create one **SurfaceForm** node per token or morpheme span.
2. For each analyzer hypothesis, create a hypothesis node with score s_h .
3. Attach root, radical, template, binyan, feature, lemma, gloss, and span nodes. Share root and paradigm nodes across forms when the analyzer or lexicon identifies the same root.
4. Add **fills_slot** edges from radicals to template slots and from slots to surface spans.
5. Add **same_root** and **paradigm_member** edges between lexemes/forms with the same root family.
6. Preserve uncertainty by edge weights or hypothesis scores rather than deleting non-maximal analyses.

Lemma 7.4 (Typed acyclicity of local morphology subgraphs). *If paradigm-sharing edges are removed, each local root-template analysis graph is acyclic when edges are oriented from abstract causes to surface evidence.*

Proof. Order node types by the partial order

$$\text{Root, Radical, Pat, Binyan, Feat} \prec \text{AnalyzerHypothesis} \prec \text{SurfaceForm} \prec \text{Lemma, Gloss, VerseSpan}. \quad (7.5)$$

Edges in a local analysis go from an earlier type to a later type, or from a hypothesis to a surface node. Since every directed edge increases this type rank, no directed cycle can occur. Paradigm-sharing edges intentionally introduce global family structure and are excluded from the local acyclicity claim. $\square$

Lemma 7.5 (Functoriality under graph relabeling). *Let C be a deterministic converter from morphology records to typed graphs, and let π be a relabeling of source token identifiers that preserves source spans and analyzer hypotheses. Then*

$$C(\pi \cdot R) = \pi \cdot C(R). \quad (7.6)$$

Proof. The converter creates graph nodes by deterministic functions of record fields: token id, span id, root id, hypothesis id, feature id, and lexicon id. Relabeling source token ids simply relabels the nodes whose keys contain those token ids. Shared lexical nodes such as root and binyan nodes are unaffected except for incident edge endpoints. Therefore every node and edge in $C(R)$ is carried to the corresponding node and edge in $C(\pi R)$. $\square$

7.3 Hebrew morphology losses

Let p_R , p_B , p_F , and p_L denote predicted distributions over roots, binyanim, feature bundles, and lemmas. Let $A \in [0, 1]^{3 \times S}$ be a predicted radical-to-slot alignment for a three-radical root and A^*

the target alignment. Define

$$\mathcal{L}_{\text{root}} = -\log p_R(R^*), \quad (7.7)$$

$$\mathcal{L}_{\text{binyan}} = -\log p_B(B^*), \quad (7.8)$$

$$\mathcal{L}_{\text{feat}} = -\sum_{f \in \mathcal{F}} [y_f \log p_F(f) + (1 - y_f) \log(1 - p_F(f))], \quad (7.9)$$

$$\mathcal{L}_{\text{align}} = -\sum_{i=1}^3 \sum_{s=1}^S A_{is}^* \log A_{is}. \quad (7.10)$$

Paradigm consistency can be enforced by a cycle loss. If z_w is the embedding of a surface form and z_R is the embedding of its root family,

$$\mathcal{L}_{\text{par}} = \sum_{(w, w') : \text{Root}(w) = \text{Root}(w')} \|(z_w - z_R) - (z_{w'} - z_R)\|_2^2 - \sum_{(w, u) : \text{Root}(w) \neq \text{Root}(u)} \max(0, \gamma - \|z_w - z_u\|)^2. \quad (7.11)$$

Proposition 7.6 (Paradigm loss is relabeling invariant). *If root-family membership is represented by equivariant graph edges and embeddings transform by token permutation, then $\mathcal{L}_{\text{par}}$ is invariant under graph relabeling.*

Proof. Relabeling permutes the set of surface-form nodes and root-family edges but does not change which unordered pairs are same-root or different-root. The summands depend only on Euclidean distances between the corresponding permuted embeddings. Euclidean distance is unchanged by row reindexing. Therefore the entire sum is invariant. $\square$

7.4 Tropical alignment for root extraction

Root-template alignment can be cast as a shortest-path problem over a small weighted finite-state graph. Let $w_1, \dots, w_T$ be surface characters or consonantal skeleton symbols, and let template states record how many radical slots have been filled. A transition either consumes a surface symbol as affix/vowel material or aligns it to the next radical slot. With costs $c_t(q, q')$, the best analysis is

$$D_{t+1}(q') = \min_q (D_t(q) + c_t(q, q')). \quad (7.12)$$

Theorem 7.7 (Root-template alignment is min-plus dynamic programming). *For a fixed finite template inventory and bounded token length T , the best radical-slot alignment is computed by a finite min-plus circuit and can therefore be represented by tropical ToricGT layers.*

Proof. The dynamic program has finitely many states because the template inventory and maximum word length are bounded. Each update is a minimum over finitely many predecessor states of an affine expression $D_t(q) + c_t(q, q')$. This is a min-plus gate. Unrolling for T time steps yields a finite min-plus circuit. By the tropical semiring approximation theorem above, ToricGT can implement or approximate this circuit on the bounded domain. $\square$

7.5 Hebrew graph tasks

ToricGT should include at least the following Hebrew-specific tasks.

1. **Root recovery:** predict the root node and ordered radicals from a surface form and context.

Reasoning capacity is audited at each computational layer

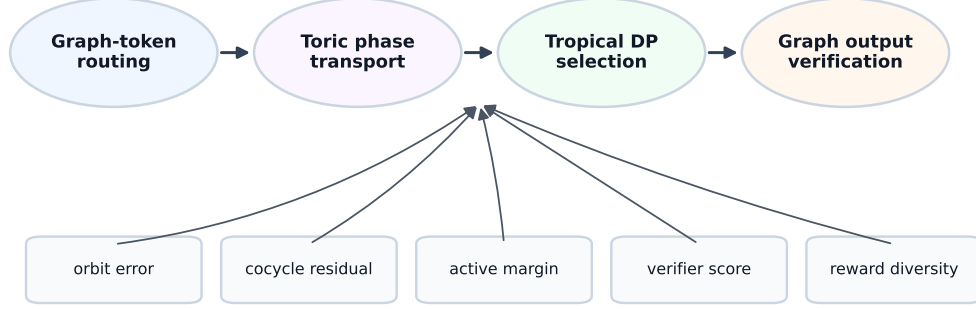


Figure 5: Reasoning capacity as a composition of graph-token routing, algebraic transport, tropical inference, and graph-level output. The model should expose checks at each stage rather than returning only an answer string.

2. **Binyan classification:** predict stem/binyan and verbal pattern.
3. **Feature tagging:** predict person, number, gender, tense/aspect, state, and part of speech when represented in the source annotation.
4. **Paradigm completion:** given several forms in a root family, predict missing forms or missing feature nodes.
5. **Graph disambiguation:** choose among analyzer hypotheses using local context and source-span dependencies.
6. **Interlinear graph construction:** convert text plus morphology into lemma, gloss, and dependency graphs.

8 Reasoning capacity and visualization contract

Definition 8.1 (Visualization contract). A ToricGT run satisfies the visualization contract if it emits, for every evaluation batch, the following diagnostic objects: relabeling orbits, attention/ring block provenance, tropical active-face maps, toric phase trajectories, Hebrew root-template sub-graphs when applicable, proof-dependency graphs, GFlowNet terminal samples, and reward/margin/error summaries.

Visualization contract dashboard



Figure 6: Visualization dashboard targets. The dashboard should make reasoning behavior falsifiable: a good scalar metric is not sufficient if equivariance, phase, active-face, morphology, or diversity diagnostics fail.

Diagnostic	Computation	Failure mode exposed
Equivariance orbit	$\ F(\pi G) - P_\pi F(G)\ $ over random π	absolute-position leakage, edge-order bugs
Tropical active face	$A_{ic} = \arg \max_j (S_{ij} + V_{jc})$ and margin Δ_{ic}	tropical collapse, unstable DP decisions
Torus commutator	$\ U_j U_k H - e^{2\pi i \Theta_{jk}} U_k U_j H\ $	phase memorization without cocycle structure
Hebrew paradigm lattice	root-family edges and feature-bundle consistency	string memorization without root-template abstraction
GFlowNet diversity	terminal graph count, edit distance, reward histogram	mode collapse, reward hacking
Polar cache residual	$\ K - \hat{K}\ $, $\ V - \hat{V}\ $, argmax preservation rate	quantization damaging long-context retrieval

Proposition 8.2 (Visualization invariance). *Suppose diagnostic functions are defined from graph-invariant quantities or equivariant token quantities followed by canonical graph rendering. Then relabeling the input graph changes only node/edge labels in the visualization, not the diagnostic values.*

Proof. Equivariance errors are norms of aligned outputs and are invariant under row permutation. Active-face sets transform by the same token permutation, and canonical rendering quotients out token labels. Commutator losses act on channel operators shared across rows, so row permutations do not change Frobenius norms. Root-family and paradigm diagnostics are computed over graph

edges whose incidence is preserved by relabeling. GFlowNet terminal graph diversity is computed up to graph edit distance or canonical graph hashes. Therefore the diagnostic values are invariant, while rendered node names may change. $\square$

9 Embedding-space GFlowNet fine-tuning

A GFlowNet samples terminal graph objects with probability proportional to reward. In ToricGT, the default Markov decision process acts in embedding space.

Definition 9.1 (Embedding-space state). A state is

$$s_t = (h_G^{(t)}, H_V^{(t)}, H_E^{(t)}, m^{(t)}), \quad (9.1)$$

where h_G is a graph embedding, H_V and H_E are node and edge latent tables, and m is metadata such as current root-analysis hypotheses, proof obligations, braid word state, or torus normal-form state.

Actions include adding a reasoning node, adding or relabeling an edge, applying a braid generator $\sigma_i^{\pm 1}$, applying an affine Coxeter reflection s_i , applying a toric shift or clock operation, choosing a tropical active face, requesting a local proof check, selecting a Hebrew analyzer hypothesis, and terminating.

Definition 9.2 (Trajectory balance loss). For a trajectory $\tau = (s_0, a_0, s_1, \dots, s_T = x)$, forward policy P_F , backward policy P_B , reward $R(x) > 0$, and learned partition scalar $Z > 0$, trajectory balance is

$$\mathcal{L}_{\text{TB}}(\tau) = \left(\log Z + \sum_{t=0}^{T-1} \log P_F(s_{t+1} \mid s_t) - \log R(x) - \sum_{t=0}^{T-1} \log P_B(s_t \mid s_{t+1}) \right)^2. \quad (9.2)$$

Theorem 9.3 (Trajectory balance terminal law). *If the trajectory-balance equation holds exactly for every complete trajectory and the backward policy is normalized over parents of every state, then the marginal probability of sampling terminal object x under the forward policy is proportional to $R(x)$.*

Proof. Exact trajectory balance states

$$Z \prod_{t=0}^{T-1} P_F(s_{t+1} \mid s_t) = R(x) \prod_{t=0}^{T-1} P_B(s_t \mid s_{t+1}). \quad (9.3)$$

Sum both sides over all trajectories ending at fixed terminal object x . The left side becomes $Z P_F(x)$, the forward marginal probability of reaching x . The right side becomes $R(x)$ times the sum of backward probabilities over all reverse paths from x to the initial state. Because P_B is normalized over parents and every reverse path terminates at the unique initial state, this sum is 1. Hence $P_F(x) = R(x)/Z$. $\square$

A useful ToricGT reward is

$$R(x) = \exp \left(\frac{\lambda_c C(x) + \lambda_n N(x) - \lambda_e E_{\text{eq}}(x) + \lambda_t \Delta_{\text{trop}}(x) - \lambda_H H(x) - \lambda_\Theta E_\Theta(x)}{\tau} \right), \quad (9.4)$$

where C is correctness, N is novelty, E_{eq} is equivariance error, Δ_{trop} is tropical margin, H is complexity, and E_Θ is toric commutator error.

10 Interleaving PolarQuant with ToricGT ring attention

10.1 Applicability

PolarQuant is directly relevant when ToricGT performs long-context autoregressive graph-token decoding, verifier decoding over long proof graphs, or ring-attention inference over very long tokenized graph traces. It is less relevant to small full-context training batches where no KV cache is materialized or where training memory is dominated by activations rather than cached keys and values. Its best role in ToricGT is therefore optional: a long-context inference and evaluation module, plus a stress-test for ring attention under compressed cache memory.

Han, Kacham, Karbasi, Mirrokni, and Zandieh introduce PolarQuant as a KV-cache quantization method using random preconditioning, recursive polar transformation, and angle quantization [18]. The method is motivated by the fact that KV cache memory scales with context length, layers, and heads; their paper reports over $\times 4.2$ compression while maintaining strong long-context performance.

10.2 Recursive polar transform

Assume the head dimension d_h is a power of two. For $x \in \mathbb{R}^{d_h}$, define level-one angles and radii by pairing coordinates:

$$r_j^{(1)} = \sqrt{x_{2j-1}^2 + x_{2j}^2}, \quad \psi_j^{(1)} = \text{atan2}(x_{2j}, x_{2j-1}), \quad j = 1, \dots, d_h/2. \quad (10.1)$$

For $\ell \geq 2$, recursively pair radii:

$$r_j^{(\ell)} = \sqrt{(r_{2j-1}^{(\ell-1)})^2 + (r_{2j}^{(\ell-1)})^2}, \quad \psi_j^{(\ell)} = \tan^{-1} \left(\frac{r_{2j}^{(\ell-1)}}{r_{2j-1}^{(\ell-1)}} \right). \quad (10.2)$$

The transform stores one final radius $r^{(\log_2 d_h)}$ and $d_h - 1$ angles.

Lemma 10.1 (Polar transform is invertible off coordinate-pair degeneracies). *For $x \neq 0$, the recursive polar representation with exact radius and exact angles reconstructs x uniquely, up to the standard angle-convention choices on zero coordinates.*

Proof. At level ℓ , each pair (a, b) is represented by $r = \sqrt{a^2 + b^2}$ and angle ψ satisfying $a = r \cos \psi$, $b = r \sin \psi$ with the level-appropriate quadrant convention. Starting from the top radius, recursively apply this inverse map to recover lower-level radii and finally the original coordinate pairs. If a pair has zero radius, its angle is irrelevant; fixing a deterministic convention yields uniqueness of the reconstructed vector. $\square$

Lemma 10.2 (Gaussian preconditioning gives predictable radii). *Let $S \in \mathbb{R}^{m \times d}$ have i.i.d. $\mathcal{N}(0, 1)$ entries. For fixed $x \in \mathbb{R}^d$, $Sx \sim \mathcal{N}(0, \|x\|_2^2 I_m)$.*

Proof. The i th coordinate of Sx is $\sum_j S_{ij}x_j$, a linear combination of independent centered Gaussians. Hence it is Gaussian with mean zero and variance $\sum_j x_j^2 = \|x\|_2^2$. Distinct rows of S are independent, so the coordinates are independent. Therefore the covariance matrix is $\|x\|_2^2 I_m$. $\square$

Proposition 10.3 (Angle concentration heuristic). *After random preconditioning, recursive polar angles at higher levels concentrate around $\pi/4$ because they compare norms of two independent high-dimensional Gaussian subvectors of equal dimension.*

PolarQuant-style cache interleaved with ring attention

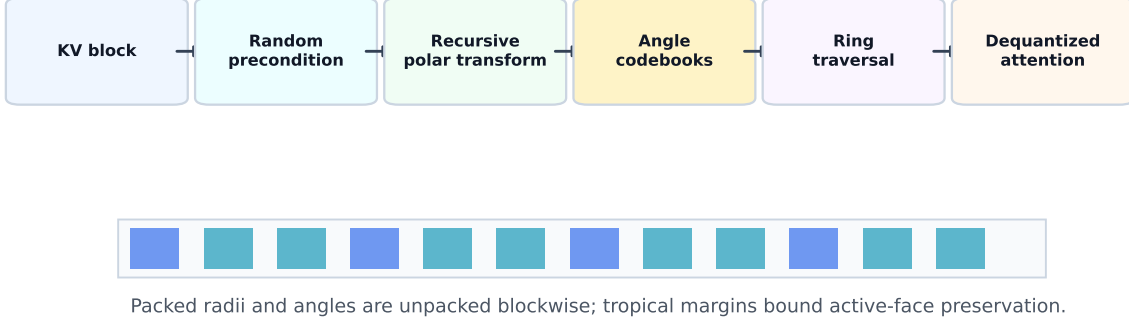


Figure 7: PolarQuant interleaving with ring attention. Each key-value block is randomly preconditioned, recursively transformed to polar coordinates, angle-quantized, packed, and dequantized only when the block reaches a query device.

Proof. At a higher recursive level, an angle has form

$$\psi = \tan^{-1} \left(\frac{R_2}{R_1} \right), \quad (10.3)$$

where R_1 and R_2 are norms of independent Gaussian subvectors of the same dimension m . Each R_i^2 is chi-square distributed up to scale, with mean proportional to m and variance proportional to m . Thus $R_i/\sqrt{m}$ concentrates around the same constant as m grows. Consequently R_2/R_1 concentrates around 1, and ψ concentrates around $\tan^{-1}(1) = \pi/4$. PolarQuant formalizes this distribution and uses it for codebook design [18]. $\square$

10.3 Polar ring cache

Definition 10.4 (Polar ring cache). For each ring block B_r , head h , and layer ℓ , store quantized polar representations

$$\text{PQ}(K_{\ell h, B_r}), \quad \text{PQ}(V_{\ell h, B_r}) \quad (10.4)$$

instead of full-precision K, V . During ring traversal, a block is dequantized locally or fused into a kernel that computes $Q\hat{K}^\top$ and attention-value products without materializing full $\hat{K}, \hat{V}$ globally.

Lemma 10.5 (Score perturbation from quantized keys). Assume $\|q_i\|_2 \leq R_Q$ for every query and $\|k_j - \hat{k}_j\|_2 \leq \epsilon_K$ for every key. Then

$$\|S - \hat{S}\|_\infty \leq \frac{R_Q \epsilon_K}{\sqrt{d_h}}. \quad (10.5)$$

Proof. For every pair (i, j) ,

$$\left| S_{ij} - \widehat{S}_{ij} \right| = \frac{1}{\sqrt{d_h}} \left| q_i^\top (k_j - \widehat{k}_j) \right| \leq \frac{1}{\sqrt{d_h}} \|q_i\|_2 \|k_j - \widehat{k}_j\|_2 \leq \frac{R_Q \epsilon_K}{\sqrt{d_h}}, \quad (10.6)$$

by Cauchy-Schwarz. $\square$

Theorem 10.6 (Tropical output stability under PolarQuant cache). *Assume key and value dequantization errors obey*

$$\|k_j - \widehat{k}_j\|_2 \leq \epsilon_K, \quad \|v_j - \widehat{v}_j\|_\infty \leq \epsilon_V, \quad (10.7)$$

and $\|q_i\|_2 \leq R_Q$. Then tropical attention with dequantized PolarQuant cache satisfies

$$\|Y - \widehat{Y}\|_\infty \leq \frac{R_Q \epsilon_K}{\sqrt{d_h}} + \epsilon_V. \quad (10.8)$$

If the full-precision tropical margin is larger than twice this bound, the active key is unchanged.

Proof. The previous lemma bounds score perturbation by $\epsilon_S = R_Q \epsilon_K / \sqrt{d_h}$. Value perturbation is bounded by ϵ_V . Apply the Lipschitz and argmax-stability lemma for tropical attention. $\square$

Proposition 10.7 (Equivariance of a shared PolarQuant cache). *If the random preconditioner, codebooks, and quantization rules are shared over token rows and ring blocks, and block schedules are label-independent or transformed consistently, then PolarQuant-dequantized ring attention remains graph equivariant up to numerical quantization error.*

Proof. The preconditioner and quantizer act identically on every key/value row. Therefore quantizing after a row permutation gives the same packed rows in permuted order, modulo deterministic tie-breaking. Ring traversal is a schedule; if it is label-independent or permuted with the tokens, it does not introduce semantic label dependence. Dequantized attention is equivariant by the attention equivariance proof, with values replaced by their quantized approximations. The only deviation from full-precision equivariance comes from finite-precision quantization and tie-breaking. $\square$

10.4 Memory accounting

Let B_{fp} be bits per full-precision coordinate, typically 16 for bf16/fp16 KV caches. If a polar cache stores one radius using B_r bits and angle level ℓ uses b_ℓ bits per angle, then amortized bits per coordinate are approximately

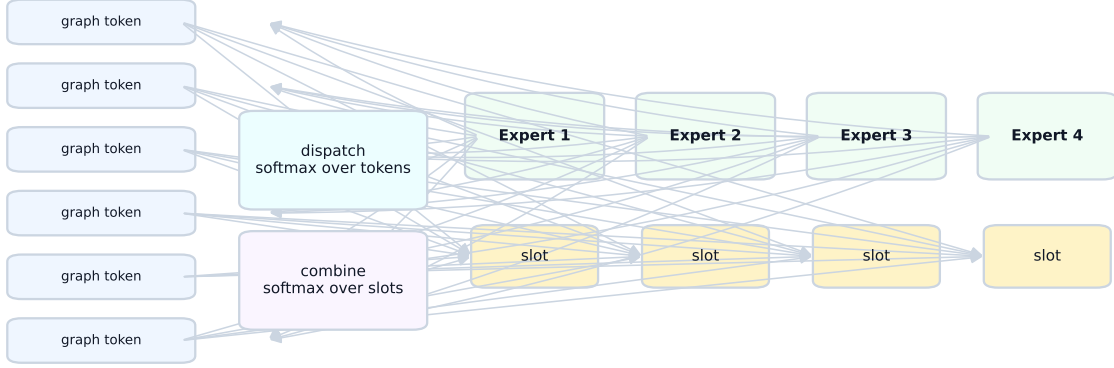
$$B_{\text{polar}} \approx \frac{B_r + \sum_{\ell=1}^{\log_2 d_h} (d_h / 2^\ell) b_\ell}{d_h} + B_{\text{overhead}}, \quad (10.9)$$

where B_{overhead} accounts for packed-index alignment and amortized codebook storage. The compression ratio is

$$\rho_{\text{cache}} \approx \frac{B_{\text{fp}}}{B_{\text{polar}}}. \quad (10.10)$$

Because codebooks are shared and preconditioning avoids per-block scale/zero-point storage, the overhead can be lower than blockwise affine quantization.

Default 4-expert Soft-MoE graph-token block



Shared routing preserves permutation equivariance; diagnostics report expert mass and slot entropy.

Figure 8: Soft-MoE ToricGT block. Graph tokens are softly dispatched into slot mixtures, typed experts process the slots, and combine weights project expert outputs back to the original token stream. The same equations handle node, edge, proof, Hebrew, tropical, and toric tokens when masks and type channels are equivariant.

11 Soft-MoE ToricGT blocks

11.1 Motivation and placement

Soft-MoE is useful for ToricGT for a different reason than ordinary large-scale MoE. The goal is not merely to increase parameter count. The goal is to allocate typed computation to different graph-token roles without hard token dropping or brittle top- k routing. In a ToricGT block, experts may specialize as generic feed-forward maps, tropical dynamic-programming gates, toric phase maps, Hebrew root-template analyzers, local proof verifiers, or graph-decoder refiners. Because Soft-MoE computes differentiable weighted mixtures rather than discrete assignments, the routing map remains smooth and can be made permutation equivariant.

Puigcerver, Riquelme, Mustafa, and Houlsby define Soft-MoE as a fully differentiable MoE mechanism where each expert slot receives a weighted average of all input tokens and the output tokens are reconstructed as weighted averages of the expert outputs [14]. The public PyTorch implementation referenced for this project exposes a `SoftMoE` layer with arguments such as `in_features`, `out_features`, `num_experts`, and `slots_per_expert` [17]. ToricGT uses that mechanism as a graph-token feed-forward replacement, not as a black-box vision-only module.

11.2 Equations

Let $X \in \mathbb{R}^{B \times L \times d}$ be a batch of graph-token hidden states and let $M \in \{0, 1\}^{B \times L}$ be the validity mask. A Soft-MoE layer has E experts and S slots per expert. Let $\Phi \in \mathbb{R}^{d \times E \times S}$ be learned slot vectors and let $g > 0$ be a learned temperature. Use normalized vectors

$$\bar{x}_{bi} = \frac{x_{bi}}{\|x_{bi}\|_2 + \epsilon}, \quad \bar{\phi}_{es} = \frac{\phi_{es}}{\|\phi_{es}\|_2 + \epsilon}, \quad (11.1)$$

and define token-slot logits

$$A_{bies} = g \langle \bar{x}_{bi}, \bar{\phi}_{es} \rangle. \quad (11.2)$$

Invalid tokens are removed by setting $A_{bies} = -\infty$ whenever $M_{bi} = 0$. Dispatch weights normalize over tokens for each slot:

$$D_{bies} = \frac{\exp(A_{bies})M_{bi}}{\sum_{j=1}^L \exp(A_{bj es})M_{bj} + \epsilon}. \quad (11.3)$$

Combine weights normalize over slots for each token:

$$C_{bies} = \frac{\exp(A_{bies})}{\sum_{e'=1}^E \sum_{s'=1}^S \exp(A_{bie' s'}) + \epsilon}. \quad (11.4)$$

The input slot for expert e and slot s is

$$Z_{bes} = \sum_{i=1}^L D_{bies} X_{bi}. \quad (11.5)$$

Each expert $f_e : \mathbb{R}^d \rightarrow \mathbb{R}^d$ processes its slots:

$$\tilde{Z}_{bes} = f_e(Z_{bes}). \quad (11.6)$$

The output token is reconstructed by

$$Y_{bi} = \sum_{e=1}^E \sum_{s=1}^S C_{bies} \tilde{Z}_{bes}. \quad (11.7)$$

A residual gate $\alpha_\ell \in [0, 1]$ is used in practice:

$$X_{\ell+1} = X_\ell + \alpha_\ell \text{Dropout}(Y_\ell), \quad \alpha_\ell \text{ initialized near } 0.05 \text{ or } 0.1. \quad (11.8)$$

This conservative initialization prevents the expert bank from dominating early training before the graph-token features have useful semantics.

Lemma 11.1 (Soft-MoE graph-token equivariance). *Let P be a token permutation matrix that preserves the validity mask by $M(PX) = PM(X)$. Suppose Φ , the experts f_e , and the temperature g are shared across token rows and do not depend on absolute token indices. Then*

$$\text{SoftMoE}(PX, PM) = P \text{SoftMoE}(X, M). \quad (11.9)$$

Proof. The logits satisfy

$$A(PX)_{ies} = A(X)_{\pi^{-1}i, e, s}. \quad (11.10)$$

The dispatch denominator for a fixed (e, s) sums over all valid tokens, so it is invariant under reindexing. Therefore

$$D(PX)_{ies} = D(X)_{\pi^{-1}i, e, s}. \quad (11.11)$$

The slot mixture is unchanged:

$$\begin{aligned} Z(PX)_{es} &= \sum_i D(PX)_{ies} (PX)_i = \sum_i D(X)_{\pi^{-1}i, e, s} X_{\pi^{-1}i} \\ &= \sum_j D(X)_{jes} X_j = Z(X)_{es}. \end{aligned} \quad (11.12)$$

Thus expert outputs $\tilde{Z}_{es}$ are unchanged. The combine weights satisfy $C(PX)_{ies} = C(X)_{\pi^{-1}i,e,s}$ because their denominator normalizes only over slots for the same token. Hence

$$Y(PX)_i = \sum_{e,s} C(X)_{\pi^{-1}i,e,s} \tilde{Z}_{es} = Y(X)_{\pi^{-1}i}, \quad (11.13)$$

which is exactly $PY(X)$. $\square$

Proposition 11.2 (Soft-MoE preserves TokenGT equivariance). *Replacing any shared token-wise feed-forward block of an equivariant ToricGT layer by the Soft-MoE block above preserves graph-to-graph equivariance.*

Proof. The surrounding attention, normalization, residual, and decoder maps are equivariant by earlier results. The Soft-MoE block is equivariant by the preceding lemma. Compositions and residual sums of equivariant maps are equivariant. $\square$

11.3 Typed expert bank for ToricGT

For the full 8M-35M graph-reasoning model, use a small typed bank rather than hundreds of experts.

Expert	Function	Practical notes
Generic MLP	ordinary dense nonlinear map	always present; fallback for arbitrary continuous approximation
Tropical DP	max/min-plus candidate scoring and margin features	use for shortest path, Viterbi, active-face, and transitive-closure tasks
Toric phase	clock/shift/cocycle channels and phase normalization	regularize lightly; avoid forcing phase structure on arbitrary data
Hebrew root	radical alignment, template/binyan features, paradigm consistency	useful only for Hebrew or Semitic morphology batches; can be gated by type embeddings
Proof verifier	local step validity and dependency consistency	attach to equation/proof-obligation nodes
Graph decoder	node/edge graph-to-graph refinement	useful near output layers

Recommended defaults are $E = 4-8$ experts, $S = 1-4$ slots per expert, g initialized near 1, and Soft-MoE enabled by default in the upper half of the network. For $L \leq 2048$, $ES \leq 32$ is normally safe on a 4090. For longer graph traces, use chunked dispatch: compute A blockwise over tokens, accumulate dispatch denominators by log-sum-exp, then form slots in a second streaming pass. This keeps the slot computation compatible with ring attention.

11.4 Implementation details

A minimal PyTorch implementation for graph tokens has the following shape discipline.

```
# X: [B, L, d], mask: [B, L]
Xn = l2_normalize(X, dim=-1)
```

```

Pn = l2_normalize(Phi, dim=0) # [d, E, S]
logits = scale * torch.einsum('bld,des->bles', Xn, Pn)
logits = logits.masked_fill(~mask[:, :, None, None], -torch.inf)
D = torch.softmax(logits, dim=1) # dispatch over tokens
C = torch.softmax(logits.flatten(2), dim=-1).view(B, L, E, S)
slots = torch.einsum('bld,bles->besd', X, D)
out_slots = stack([experts[e](slots[:, e]) for e in range(E)], dim=1)
Y = torch.einsum('besd,bles->bld', out_slots, C)
X = X + residual_gate * dropout(Y)

```

The implementation should explicitly test: output shape, mask handling, gradient finiteness, deterministic behavior under token permutation, absence of NaNs when all tokens of a batch element are padded, and agreement between chunked and unchunked dispatch within numerical tolerance.

11.5 Prefix-causal Soft-MoE for language-model scoring

The graph-to-graph ToricGT encoder is allowed to use bidirectional graph context. The Parameter-Golf micro language model is not. A direct Soft-MoE implementation that dispatches each expert slot using all tokens in a sequence is noncausal: the slot representation for position t would include future tokens $x_{>t}$ and would therefore leak validation information. The contest-facing model must use a prefix-causal version of Soft-MoE whenever the output at token t is scored autoregressively.

Flatten the expert and slot indices into $s \in \{1, \dots, ES\}$ and let a_{ts} be the token-slot logit. For an autoregressive sequence $x_1, \dots, x_T$, define strict-prefix dispatch accumulators

$$N_s^{(t)} = \sum_{m < t} \exp(a_{ms}) x_m, \quad (11.14)$$

$$R_s^{(t)} = \sum_{m < t} \exp(a_{ms}), \quad (11.15)$$

$$Z_s^{(t)} = \frac{N_s^{(t)}}{R_s^{(t)} + \epsilon}. \quad (11.16)$$

The expert output available at position t is

$$\tilde{Z}_s^{(t)} = f_{e(s)}(Z_s^{(t)}), \quad (11.17)$$

and the causal combine weights are normalized only over slots for the current token,

$$C_{ts} = \frac{\exp(a_{ts})}{\sum_{u=1}^{ES} \exp(a_{tu}) + \epsilon}. \quad (11.18)$$

The Soft-MoE residual used to predict token x_{t+1} is then

$$y_t = \sum_{s=1}^{ES} C_{ts} \tilde{Z}_s^{(t)}. \quad (11.19)$$

After the loss for position t has been computed, the accumulators may be updated by

$$N_s^{(t+1)} = N_s^{(t)} + \exp(a_{ts}) x_t, \quad (11.20)$$

$$R_s^{(t+1)} = R_s^{(t)} + \exp(a_{ts}). \quad (11.21)$$

This is a score-before-update rule. It is the Soft-MoE analogue of ordinary causal attention: the current prediction can use only the strict prefix.

Lemma 11.3 (Prefix-causal Soft-MoE is valid for autoregressive BPB). *For every position t , the output y_t of the prefix-causal Soft-MoE block is a measurable function of $x_{<t}$, the current unscored embedding used to choose a distribution over x_t only when that embedding itself is derived from $x_{<t}$, and the fixed model parameters. In particular, if the surrounding attention and embedding pipeline is causal, replacing a dense feed-forward block by prefix-causal Soft-MoE preserves strict left-to-right scoring.*

Proof. By construction, $N_s^{(t)}$ and $R_s^{(t)}$ are sums over indices $m < t$. Therefore $Z_s^{(t)}$ and $\tilde{Z}_s^{(t)}$ are functions only of the strict prefix and the parameters of f_e . The combine weights C_{ts} may depend on the hidden state at t , but in a causal language model that hidden state is produced from the same strict prefix. Hence y_t is a function of the strict prefix. The update with x_t occurs only after scoring position t , so no scored probability can depend on its own target or on future targets. $\square$

A vectorized training implementation can compute these quantities by prefix scans. In PyTorch, the noncausal dispatch

```
D = softmax(logits, dim=1)          # invalid for LM scoring if dim=1 sees future
slots = einsum('bld,bles->besd', X, D)
```

should be replaced in the contest model by cumulative sums over time:

```
w = torch.exp(logits.flatten(2)).clamp_max(1e4)      # [B, T, ES]
prefix_num = torch.cumsum(w[..., None] * X[:, :, None, :], dim=1)
prefix_den = torch.cumsum(w, dim=1)
# strict prefix: shift right before forming slots
N = torch.cat([zeros_like(prefix_num[:, :1]), prefix_num[:, :-1]], dim=1)
R = torch.cat([zeros_like(prefix_den[:, :1]), prefix_den[:, :-1]], dim=1)
slots_t = N / (R[..., None] + eps)                  # [B, T, ES, d]
out_slots_t = experts(slots_t)
C = torch.softmax(logits.flatten(2), dim=-1)
Y = torch.einsum('bts,btsd->btd', C, out_slots_t)
```

For the graph-reasoning encoder this causal restriction is unnecessary; for the Parameter-Golf track it is mandatory.

11.6 Soft-MoE regularizers and diagnostics

Soft-MoE should be monitored rather than trusted. Define expert combine mass

$$m_e = \frac{1}{BL} \sum_{b,i,s} C_{bies}, \quad (11.22)$$

slot dispatch entropy

$$H_{es}^D = - \sum_i D_{bies} \log(D_{bies} + \epsilon), \quad (11.23)$$

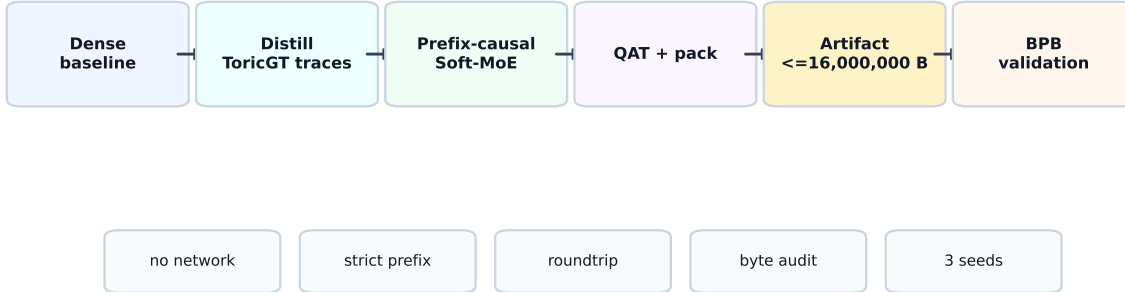
and token combine entropy

$$H_i^C = - \sum_{e,s} C_{bies} \log(C_{bies} + \epsilon). \quad (11.24)$$

A light usage regularizer is

$$\mathcal{L}_{\text{moe}} = \lambda_{\text{bal}} \sum_e \left(m_e - \frac{1}{E} \right)^2 - \lambda_D \mathbb{E}[H^D] + \lambda_C \mathbb{E}[\max(0, H^C - H_{\text{max}})]. \quad (11.25)$$

Parameter-Golf compression track



The contest LM is a separate deployment artifact; the full ToricGT graph model is the teacher and diagnostic scaffold.

Figure 9: Parameter-Golf protocol for ToricGT. The full graph-to-graph model remains the research model. The contest-facing model is a micro language model distilled from graph-reasoning traces, equipped with byte-safe Soft-MoE adapters, trained under the wallclock budget, and exported as a single self-contained artifact with exact byte accounting and prequential BPB validation.

The sign convention encourages noncollapsed dispatch while allowing combine weights to become sharp only when the task supports specialization. Do not over-regularize m_e on mixed-domain batches: if a batch contains only Hebrew morphology, the Hebrew-root expert should be allowed to receive more mass.

12 Parameter-Golf constrained ToricGT

12.1 Contest constraints as engineering axioms

The OpenAI Model Craft Challenge: Parameter Golf asks participants to train the best language model that fits in a self-contained 16,000,000 byte artifact, trains in roughly ten minutes on the prescribed 8xH100 evaluation setting, and is scored by tokenizer-agnostic bits per byte on FineWeb validation data [15]. The ToricGT implementation should track both the official repository and the working fork used for this project [16]. The important engineering point is that the counted artifact is not a vague parameter count: it is the serialized code, compressed model state, tokenizer or byte model, and metadata that the evaluator receives. The contest setting also requires evaluation to be self-contained, with no network calls, no external downloads, no hidden validation side information, and no dependence on a training dataset at evaluation time [15]. Record-style submissions should include a runnable `train_gpt.py`, `README`, `submission.json`, training logs from independent runs, and exact byte accounting. SOTA claims require a statistically meaningful improvement over the current record, so a ToricGT-derived submission must be treated as an auditable compression experiment rather than an informal model sketch. The working fork used for ToricGT experimentation is treated as the project-local contest scaffold, while the official OpenAI repository remains the normative source for rules and compliance checks [16].

Definition 12.1 (Parameter-Golf artifact budget). Let

$$B_{\text{art}} = B_{\text{code}} + B_{\text{weights}} + B_{\text{tokenizer}} + B_{\text{metadata}}. \quad (12.1)$$

A Parameter-Golf-compliant artifact must satisfy

$$B_{\text{art}} \leq 16,000,000. \quad (12.2)$$

For a model with tensors W_r quantized to b_r bits per scalar and entropy-compressed by factor $\rho_r \geq 1$,

$$B_{\text{weights}} \approx \sum_r \frac{b_r |W_r|}{8\rho_r} + B_{\text{packing}}. \quad (12.3)$$

The contest objective can be written as

$$\min_{\theta, \mathcal{C}} \text{BPB}_{\text{FineWeb}}(q_{\theta, \mathcal{C}}) \quad \text{subject to} \quad B_{\text{art}} \leq 16,000,000, \quad (12.4)$$

where $\mathcal{C}$ denotes the compression/export scheme and

$$\text{BPB}(q) = -\frac{1}{N_{\text{bytes}}} \sum_t \log_2 q(b_t \mid b_{<t}). \quad (12.5)$$

This formula is tokenizer-agnostic because the denominator is bytes, not tokens.

12.2 Prequential BPB validity

Parameter Golf scoring is best viewed as a prequential compression game. The model emits a full normalized predictive distribution, pays log loss on the next symbol, and only then may update any legal online state. Let the validation byte stream be $b_1, \dots, b_T$ and let $\ell(t)$ be the exact number of bytes represented by the tokenizer symbol scored at step t . A scoring procedure is valid if it satisfies four conditions:

1. **strict prefix dependence:** q_t depends only on $b_{<t}$, fixed parameters, and legal online state derived from already scored bytes;
2. **full normalization:** q_t is a normalized distribution over the next scored symbol;
3. **score before update:** the state update using b_t occurs after $-\log q_t(b_t)$ is recorded;
4. **single validation pass:** the validation stream is not permuted, repeated for tuning, or searched over by seeds.

Under these conditions the reported objective is

$$\text{BPB}_{\text{valid}} = \frac{\sum_{t=1}^T -\log_2 q_t(b_t \mid b_{<t})}{\sum_{t=1}^T \ell(t)}. \quad (12.6)$$

Proposition 12.2 (Score-before-update prevents validation leakage). *Assume q_t satisfies strict prefix dependence and the online state update has the form $s_{t+1} = U(s_t, b_t)$ after the loss term at t is recorded. Then the cumulative validation loss is a proper left-to-right log score and no term in the sum can depend on future validation bytes.*

Proof. We prove by induction that s_t is a function only of $b_{<t}$ and fixed parameters. The base state s_1 is fixed. If s_t is a function only of $b_{<t}$, then q_t is also a function only of $b_{<t}$ by strict prefix dependence. The loss term $-\log q_t(b_t)$ is therefore scored before any access to $b_{>t}$. The update $s_{t+1} = U(s_t, b_t)$ is then a function only of $b_{\leq t}$, closing the induction. Thus no loss term uses future information. $\square$

12.3 Micro-ToricGT language-model design

The contest model should be a language model inspired by ToricGT, not the full graph-to-graph encoder. The recommended design is:

1. a byte-level or small-BPE tokenizer, with tied input/output embeddings;
2. a small number of unique Transformer blocks reused recurrently to increase effective depth without storing more block parameters;
3. lower dense blocks for stable language modeling and upper *prefix-causal* Soft-MoE blocks with very small E and S ;
4. tiny toric/tropical adapters that add phase-like and max-gated residuals without large parameter cost;
5. quantization-aware training and final int6/int8 packing plus zlib/zstd round-trip checks;
6. optional legal score-first test-time adaptation only on validation tokens already scored, never on future validation tokens.

A safe starting microarchitecture is

$$V = 1024 \text{ or } 2048, quadd = 384 \text{ or } 512, L_{\text{unique}} = 4\text{--}6, L_{\text{effective}} = 8\text{--}12, \quad (12.7)$$

with grouped-query attention, tied embeddings, recurrent block tying, and Soft-MoE only in the last two unique blocks. If P_{fp} is the uncompressed scalar count, then int8 export roughly permits $P_{\text{fp}} < 16\text{M}$ minus code overhead; int6 export roughly permits $P_{\text{fp}} < 21\text{M}$ before entropy compression; int4 export permits more parameters but is less stable unless quantization-aware training is strong. These are engineering estimates; the actual gate is the serialized artifact size.

Proposition 12.3 (Depth recurrence improves effective computation without increasing stored block parameters). *Let a block Ψ_θ be applied r times with shared parameters. The stored parameter count is $|\theta|$, while the effective computational depth is r . If Ψ_θ is token equivariant, then Ψ_θ^r is token equivariant.*

Proof. Only one copy of θ is serialized, so stored parameters do not scale with r . The composition of r copies of the same equivariant map is equivariant because for any permutation P ,

$$\Psi_\theta^r(PX) = \Psi_\theta^{r-1}(P\Psi_\theta(X)) = P\Psi_\theta^r(X) \quad (12.8)$$

by induction. □

12.4 Soft-MoE under a 16 MB artifact cap

A naive Soft-MoE can violate the artifact budget because it stores multiple expert MLPs. The contest adaptation must use one or more of the following parameter-sharing schemes.

1. **Shared base plus expert adapters:** $f_e(x) = f_0(x) + B_e\sigma(A_ex)$ with small rank $r \ll d$.
2. **Tied expert matrices:** all experts share W_1, W_2 but have expert-specific gates, biases, or low-rank deltas.
3. **Grouped experts:** experts are reused across recurrent layers and only routing vectors Φ change by layer.

4. **Quantized experts:** expert weights are trained with straight-through quantization and exported in int6/int8.
5. **Pruned slots:** keep $ES \leq 8$ or 16 for the contest model; large slot counts are inappropriate under the wallclock cap.

For a low-rank expert adapter with rank r , the incremental parameter cost per expert is approximately

$$P_{\text{adapter}} \approx 2dr + 2d_{\text{ff}}r + O(d + d_{\text{ff}}), \quad (12.9)$$

which is far smaller than a full expert cost $2dd_{\text{ff}}$. Therefore a Parameter-Golf Soft-MoE block should store one shared dense MLP plus small expert adapters. The graph-reasoning version may use full typed experts; the contest version should not unless artifact measurements prove it fits.

12.5 Distillation from ToricGT to the contest LM

The full ToricGT model can help the contest track before submission by producing training-time supervisory signals. The contest artifact should not need the teacher at evaluation. Let p_T be a trained ToricGT-derived teacher distribution over serialized graph-reasoning traces and p_S be the micro language model. A compliant training objective on allowed training data is

$$\mathcal{L}_{\text{PG}} = \mathcal{L}_{\text{LM}} + \lambda_{\text{KD}} T^2 \text{KL} \left(p_T^{(T)}(\cdot | c) \parallel p_S^{(T)}(\cdot | c) \right) + \lambda_h \|Rh_T - h_S\|_2^2 + \lambda_q \mathcal{L}_{\text{QAT}}, \quad (12.10)$$

where c ranges over training contexts, T is distillation temperature, R is a small learned projection used only during training, and $\mathcal{L}_{\text{QAT}}$ is the quantization-aware training loss. Teacher logits, if stored, must be generated from allowed training data and must not be used as hidden validation information. In the final artifact, omit the teacher, omit distillation heads, and serialize only the student.

12.6 Artifact packing and runtime implementation

A robust contest script should separate training-time tensors from serialized tensors. During training, keep fp16 or bf16 master weights and fake-quantized forward weights. At export, pack only the deployable state:

$$\mathcal{A} = \text{Pack}(\text{Quantize}(\theta_S), \text{Tokenizer}, \text{Config}, \text{DecodeCode}). \quad (12.11)$$

The export test is not complete until the model is loaded from $\mathcal{A}$ and evaluated without the floating-point training checkpoint. The following checks should be hard failures in `train_gpt.py`:

```
assert artifact_bytes <= 16_000_000
assert not uses_network()
assert not reads_validation_except_eval_stream()
assert roundtrip_bpb_close(float_eval_bpb, packed_eval_bpb, tolerance=0.01)
assert causal_audit_passed(model)
```

The size audit should report at least

$$(B_{\text{code}}, B_{\text{weights}}, B_{\text{tokenizer}}, B_{\text{metadata}}, B_{\text{art}}), \quad (12.12)$$

and the log should include the exact command, seed, git commit, wallclock, GPU type, tokenizer, vocabulary size, number of unique blocks, recurrence depth, number of Soft-MoE experts, slots per expert, quantization bits, validation BPB, and validation bytes. For a ToricGT-derived model, also report whether causal Soft-MoE, toric adapters, tropical gates, or teacher distillation were active.

12.7 Contest record folder checklist

A ToricGT Parameter-Golf submission should create a record folder with:

```
records/<name>/
  README.md          # architecture, training recipe, size accounting
  submission.json    # author, GitHub id, val_bpb, metadata
  train.log          # automatic training/eval log, preferably 3 seeds
  train_gpt.py       # self-contained runnable script
  requirements.txt    # only if extra packages are needed
```

Before opening a pull request, run:

```
python train_gpt.py --dry_run_size_check
python train_gpt.py --quick_train --max_steps 20
python train_gpt.py --roundtrip_quantize
python train_gpt.py --eval_only --assert_no_network
```

The script should print train loss, validation loss, validation BPB, compressed model bytes, code bytes, artifact bytes, seed, wallclock, and hardware. The size check must compute decimal bytes and fail at 16,000,000, not 16 MiB.

12.8 Pragmatic risk controls

The Parameter-Golf track has different failure modes from the research model. The most important controls are:

1. keep a boring dense baseline in the record folder before adding Soft-MoE;
2. measure artifact bytes after every architecture change;
3. verify quantized round-trip evaluation, not only floating-point evaluation;
4. maintain three-seed evidence before claiming a record-quality result;
5. keep validation handling strictly score-first and never train on future validation tokens;
6. prefer self-contained torch code over fragile external imports, even if imports are nominally allowed;
7. document every compression, tokenizer, and test-time adaptation trick in the README.

The contest leaderboard shows that successful submissions often combine small-model architecture changes, quantization, recurrent or parallel residual computation, tokenizer/data transforms, and legal test-time adaptation. ToricGT should borrow these tactics only when they are compatible with the self-contained artifact and validation rules.

13 Datasets, synthetic families, and leakage control

13.1 Training mixture

The data plan is graph-first. Every raw record is normalized into a graph record with question, answer, reasoning, metadata, stable hashes, graph JSON, token-count estimates, and deterministic split fields. The local curation plan targets chain-of-thought, tree-of-thought, graph-of-thought, mathematical, programming, OpenAI-origin GSM8K, Hebrew morphology, rabbinic

Hebrew/Aramaic-English, and Jewish Hebrew text sources. It writes Parquet outputs and split reports suitable for training and, after license review, possible Hugging Face upload.

Dataset	Selected loader/config	License observed	ToricGT role
NuminaMath-CoT	default via datasets	Apache-2.0	large math CoT to proof graphs
OpenR1-Math-220k	all via datasets	Apache-2.0	multi-trace math reasoning
Codeforces-CoTs	decontaminated	CC-BY-4.0	algorithmic/programming CoT
GSM8K	Python/editorial config	MIT	OpenAI math word problems
Hendrycks MATH	main	MIT	competition math and verifier checks
MATH-500	seven subject configs	dataset card	verifier-style benchmark
Tree-of-Thoughts 24k	default/test	Apache-2.0	explicit ToT branches
Got Math 500K	raw JSON	Apache-2.0	explicit GoT math
UniMorph Hebrew	raw JSONL stream	CC-BY-SA-3.0	Hebrew lemma/inflection graphs
Sefaria parallel pairs	raw UniMorph TSV	CC-BY-4.0	rabbinic Hebrew/Aramaic-English parallel text
Sefaria Hebrew library	fallback	GPL-3.0	large Jewish Hebrew text library
Superior-Reasoning gpt-oss	default	CC-BY-4.0	frontier open-weight reasoning distillation
GPT-OSS math distill	stage1/stage2	Apache-2.0	gpt-oss math rationales
Algorithmic SFT v1	default	MIT	procedural algorithmic traces
Graph reasoning messages	default	Apache-2.0	graph-structured reasoning dialogue
DAG reasoning	default	Apache-2.0	dependency-aware reasoning traces
Opus reasoning 24k	default	Apache-2.0	high-quality reasoning style transfer

The important loader details are pragmatic. The UniMorph Hub repository may contain an old dataset script, so the Hebrew file should be downloaded directly from the UniMorph GitHub source when the current **datasets** package rejects the script. The GoT and ToT sources are raw JSON/JSONL rather than conventional multi-shard builders, so the curation script downloads and streams them directly. The Hebrew/Jewish-text slice intentionally uses Sefaria and UniMorph Hebrew sources rather than Christian-branded biblical-language datasets when the experimental goal is Jewish Hebrew/rabbinic coverage.

Hebrew rows are niqqud-aware. The curation policy is to preserve pointed Hebrew when the upstream source provides it, prefer same-consonant pointed variants when an equivalent field is present, and explicitly flag unpointed Hebrew rather than synthesize vowels into attested source text. The current audit finds 3,564,756 rows containing Hebrew, of which 2,868 are pointed upstream rows and 3,561,888 are unpointed upstream rows. Strict pointed-Hebrew runs should filter out records with `has_hebrew_without_niqqud=true`; broader Jewish-text pretraining can retain them while making the missing-vowel status visible to the model and to evaluation reports.

13.2 Normalized record schema

Every curated row should contain:

```
record_id, dataset, config, source_split, source_index,  
task_family, language, license, role,  
question, answer, solution, reasoning,  
metadata_json, text, graph_json,  
content_hash, group_hash, split,  
estimated_tokens, quality_flags_json
```

The schema separates the human-readable training view from the graph view. The `text` field supports ordinary language-model distillation and debugging; the `graph_json` field supports TokenGT conversion, graph-to-graph prediction, GFlowNet state construction, and visualization.

13.3 Graph conversion policy

The conservative base graph has a `problem` node, zero or more `reasoning_step` nodes, an `answer` node, `depends_on` edges between consecutive or referenced steps, and a `supports_answer` edge from the final step or problem node to the answer node. Dataset-specific additions are:

1. Codeforces rows add `problem_statement`, `editorial`, `public_test`, and `code_solution` nodes when available.
2. OpenR1 rows add one generation node per sampled reasoning trace when multiple traces are present.
3. UniMorph rows add `lemma`, `form`, and `features` nodes.
4. Sefaria parallel rows add `ref`, `hebrew_or_aramaic`, `english`, and `category` nodes.
5. Sefaria Hebrew-library rows add `ref`, `category`, `version`, and `text` nodes when metadata is available.
6. ToT/GoT rows preserve branch markers and tag records as `tot` or `got_math` for richer later conversion.

This representation is deliberately conservative: it is sufficient for TokenGT pretraining and can be refined later without redownloading raw sources.

13.4 Leakage-resistant splitting

Random record-level splits are invalid for this setting. Near-duplicate chain-of-thought traces, identical proof graphs, inverse braid words, conjugate torus words, repeated Hebrew lexeme paradigms, and copied Sefaria references can leak structure.

Definition 13.1 (Leakage cluster). A leakage cluster is an equivalence class generated by any of the following high-similarity relations: exact normalized-content hash, source-family key collision, text shingle SimHash prefix match, graph Weisfeiler-Lehman sketch match, algebraic normal-form equivalence, inverse/conjugate word equivalence, same Hebrew root paradigm, Sefaria reference identity, or graph-isomorphism equivalence for synthetic dynamic-programming instances.

The practical splitter computes canonical text, exact `content_hash`, task-family keys such as Codeforces problem id or Sefaria reference, and a stable SimHash prefix over word shingles. The `group_hash` combines task-family key, content hash, and SimHash prefix. Split assignment is deterministic:

$$\text{split}(g) = \begin{cases} \text{train}, & h(g) \in [0, 80), \\ \text{validation}, & h(g) \in [80, 90), \\ \text{test}, & h(g) \in [90, 100), \end{cases} \quad (13.1)$$

where $h(g)$ is a stable hash percentile. All rows with the same group key go to the same split. For synthetic algebra, stronger family holdouts override percentile splitting: hold out denominator intervals q , word lengths, Coxeter ranks, graph sizes, root families, and root-template pairs.

Proposition 13.2 (Cluster splitting prevents direct duplicate leakage). *If every leakage cluster is assigned wholly to one split, then no train/test pair is connected by the chosen leakage relations.*

Proof. Assume for contradiction that a train record r and a test record s are connected by a leakage relation. By definition, r and s lie in the same equivalence class generated by such relations. Cluster splitting assigns the entire class to a single split, contradicting that r is train and s is test. $\square$

13.5 Output layout and commands

The curation output should have the following structure.

```
data/
  curated/
    manifest.json
    split_report.json
    split_report.md
    all.parquet
    train.parquet
    validation.parquet
    test.parquet
    by_dataset/<safe_dataset_name>.parquet
  raw/hf/
    downloaded raw JSON/JSONL fallback files
```

Run a full curation with:

```
conda run -n tokengt env PYTHONPATH=src python scripts/curate_datasets.py \
  --output-dir data/curated \
  --raw-dir data/raw/hf \
  --chunk-size 20000 \
  --num-workers 16 \
  --normalize-batch-size 256
```

Run a bounded sample curation with:

```
conda run -n tokengt env PYTHONPATH=src python scripts/curate_datasets.py \
  --output-dir data/curated_sample \
  --raw-dir data/raw/hf \
  --max-records-per-source 1000 \
  --num-workers 4 \
  --normalize-batch-size 128
```

Validate outputs with:

```
conda run -n tokengt env PYTHONPATH=src python scripts/inspect_curated_data.py \
  --curated-dir data/curated
```

Use 8–16 CPU workers on a 32-thread workstation, lowering the count when other CPU-heavy jobs are running. Large direct downloads can use `aria2c` when available, but the bottleneck is often Python-side graph normalization, SimHash grouping, and Parquet writing.

13.6 Expected scale and license handling

The first full pass is expected to produce about 5.2M records: roughly 859k NuminaMath, 225k OpenR1, 500k GoT Math, 9.8k Codeforces, 8.8k GSM8K, 12.5k MATH, 24.7k ToT, 3.55M Sefaria Hebrew-library records, 3.7k Sefaria parallel pairs, and a small UniMorph Hebrew table. This is enough for the lower Chinchilla-style token budget for 8M–35M parameters once reasoning text, Hebrew text, and graph tokens are counted.

Before publishing any normalized data, re-check every license, preserve source attribution columns, and consider publishing only metadata and graph-conversion outputs where redistribution is allowed. Do not publish records from sources whose terms prohibit redistribution or require gating. GPL-licensed data, in particular, should be handled separately from permissively licensed training mixtures when downstream model or dataset release terms matter.

13.7 Synthetic generator specifications

Rotation words. Sample words in $\{U, U^{-1}, V, V^{-1}\}$, reduce to $\omega^c U^a V^b$, and emit each rewrite as a graph edge labeled `commutes_with_phase` or `cancels`. Hold out denominator intervals q , word lengths, and phase exponents.

Higher tori. Sample skew matrices Θ and words in $U_1^{\pm 1}, \dots, U_n^{\pm 1}$. Targets include ordered exponent vector $a \in \mathbb{Z}^n$ and symbolic phase

$$\exp \left(2\pi i \sum_{j < k} c_{jk} \Theta_{jk} \right). \quad (13.2)$$

Hebrew roots. Sample root families and templates from curated lexica or analyzer outputs. Create graph records with shared root nodes, radical nodes, template nodes, binyan nodes, and uncertain hypotheses. Hold out entire roots or entire root-template combinations to test abstraction.

Tropical algorithms. Generate weighted graphs and full Bellman-Ford/Floyd-Warshall traces. Record selected and dominated candidates, active faces, margins, and final distances.

14 Model, losses, and training schedule

14.1 Architecture

A practical 8M–35M parameter ToricGT family uses $L = 6$ –12 layers, width $d = 256$ –512, eight heads, MLP expansion four, softmax/hybrid lower layers, tropical ring middle and upper layers, node/edge/graph heads, optional Hebrew morphology heads, optional GFlowNet policy heads, default Soft-MoE feed-forward replacements in the upper half of the network, and optional PolarQuant

cache at inference. The full research model is graph-to-graph. The Parameter-Golf variant is a distilled micro language model with tied embeddings, recurrent blocks, quantization-aware training, and a small Soft-MoE adapter bank.

The rough parameter count is

$$P \approx L(4d^2 + 8d^2) + O(dV_{\text{feat}}), \quad (14.1)$$

where $4d^2$ is attention projection scale and $8d^2$ is a two-layer feed-forward block with expansion four. Because graph-token feature vocabularies are small compared with language vocabularies, most parameters sit in the encoder. If a dense feed-forward block is replaced by full Soft-MoE experts, the stored parameter count grows roughly like E times the expert MLP size plus dES router parameters. For the research model this can be acceptable when E is small. For Parameter Golf, use shared experts or low-rank expert adapters so that increased specialization does not destroy the artifact budget.

14.2 Composite objective

The supervised objective is

$$\begin{aligned} \mathcal{L}_{\text{sup}} = & \lambda_V \mathcal{L}_V + \lambda_E \mathcal{L}_E + \lambda_G \mathcal{L}_G + \lambda_{\text{eq}} \mathcal{L}_{\text{eq}} + \lambda_{\Theta} \mathcal{L}_{\Theta} + \lambda_{\text{trop}} \mathcal{L}_{\text{margin}} \\ & + \lambda_H (\mathcal{L}_{\text{root}} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{binyan}} + \mathcal{L}_{\text{feat}} + \mathcal{L}_{\text{par}}) + \lambda_{\text{moe}} \mathcal{L}_{\text{moe}} + \lambda_{\text{cache}} \mathcal{L}_{\text{PQ}} + \lambda_{\text{KD}} \mathcal{L}_{\text{distill}}. \end{aligned} \quad (14.2)$$

The equivariance loss is

$$\mathcal{L}_{\text{eq}} = \mathbb{E}_{\pi} \|F(\pi G) - P_{\pi} F(G)\|_2^2. \quad (14.3)$$

The torus loss is

$$\mathcal{L}_{\Theta} = \sum_{j < k} \|U_j U_k H - e^{2\pi i \Theta_{jk}} U_k U_j H\|_F^2. \quad (14.4)$$

A stable tropical objective combines margin and entropy:

$$\mathcal{L}_{\text{margin}} = -\mathbb{E}[\Delta_{\text{correct}}] + \beta \mathbb{E}[\max(0, \Delta_{\text{collapse}} - H(A))]. \quad (14.5)$$

14.3 Schedule

1. **Phase A: implementation validation.** CPU-only shape tests, equivariance tests, mask tests, and exact tropical ring-vs-full comparisons.
2. **Phase B: synthetic algebra.** Rotation words, finite tori, braid/Coxeter tasks, and tropical dynamic programming.
3. **Phase C: curated reasoning graphs.** Math, algorithmic, and Hebrew morphology graph records with clustered splits.
4. **Phase D: GFlowNet fine-tuning.** Activate trajectory or subtrajectory balance over embedding-space graph refinements.
5. **Phase E: long-context cache evaluation.** Enable PolarQuant cache for long graph-token traces and compare exact KV, unquantized ring, and polar ring cache.
6. **Phase F: Parameter-Golf distillation and compression.** Distill graph-reasoning behavior into a micro language model, add compact Soft-MoE adapters only after a dense baseline is stable, run quantization-aware training, and export a self-contained artifact with exact byte accounting.

15 Evaluation and ablations

Core metrics include node accuracy/regression, edge accuracy/regression, graph answer exact match, proof validity, equivariance error, tropical margin and active-face entropy, torus commutator error, Hebrew root/binyan/feature F1, paradigm consistency, Soft-MoE expert utilization and slot entropy, Parameter-Golf artifact bytes and validation BPB for the micro-LM track, GFlowNet reward calibration, terminal diversity, and OOD generalization over graph size, word length, denominator q , Coxeter rank, and Hebrew root families.

Ablation	Question answered
Softmax TokenGT, no toric features	Is graph tokenization alone sufficient?
Softmax TokenGT + toric features	Do phase channels help without tropical heads?
Tropical heads, no ring schedule	Does max-plus reasoning help independent of systems schedule?
Tropical ring heads	Does exact blockwise evaluation preserve quality at longer context?
No equivariance loss	Does architecture alone enforce enough symmetry in practice?
No torus regularizer	Are phase channels learned structurally or memorized?
No Hebrew graph factorization	Does root-template structure outperform string-only morphology?
GFlowNet token-space vs embedding-space	Which action space gives more diverse high-reward graphs?
Dense FFN vs Soft-MoE ToricGT	Does typed differentiable routing improve graph reasoning without destabilizing training?
Full expert Soft-MoE vs shared-adaptor Soft-MoE	Which expert parameterization gives the best quality/byte tradeoff?
Full ToricGT vs micro-ToricGT LM	Which graph-reasoning inductive biases survive the Parameter-Golf compression regime?
Full KV vs PolarQuant cache	Does polar cache compression preserve long-context graph reasoning?

16 Implementation blueprint

The codebase should be organized as follows.

1. `typed_graph.py`: graph schema, relabeling, canonicalization, masks.
2. `graph_tokenizer.py`: node, edge, endpoint, toric, Hebrew, and structural feature channels.
3. `tropical_attention.py`: full tropical attention, ring tropical attention, provenance, margins.
4. `toric_algebra.py`: continued fractions, Weyl pairs, cocycles, normal forms, trace tasks.
5. `hebrew_graphs.py`: root-template graph conversion, analyzer-hypothesis graphs, paradigm graphs.

6. `model.py`: encoder, task heads, graph decoder, GFlowNet policy head.
7. `gflownet.py`: trajectory balance, subtrajectory balance, replay buffer, novelty metrics.
8. `soft_moe_toric.py`: graph-token Soft-MoE dispatch/combine, typed expert banks, slot diagnostics, chunked dispatch.
9. `causal_soft_moe.py`: prefix-scan Soft-MoE for autoregressive Parameter-Golf scoring, causal audits, score-before-update tests.
10. `polar_cache.py`: preconditioning, recursive polar transform, codebooks, packing, dequantized kernels.
11. `parameter_golf_export.py`: byte accounting, tied-weight packing, QAT export, zlib/zstd round-trip checks, record-folder scaffolding.
12. `bpb_audit.py`: byte-length accounting, validation-order checks, normalized-distribution tests, seed-bruteforce guards.
13. `visualization.py`: spiral projection, tropical cells, root graphs, braid/Coxeter trajectories, reward landscapes.
14. `leakage.py`: text shingles, graph hashes, algebraic normal forms, root-family cluster splits.

17 Discussion and limits

ToricGT is intentionally finite. It learns finite presentations, finite approximants, finite graph algorithms, finite root-template analyses, and finite proof graphs. The operator-algebraic layer supplies structure and tests, not a claim that a 35M parameter model contains $\mathcal{A}_\theta$ as an infinite C^* -algebra. The Hebrew layer supplies an explicit graph factorization of morphology, not a claim that every ambiguous biblical form has a unique analysis. The PolarQuant layer supplies a plausible and mathematically analyzable cache-compression path for long-context inference, not a claim that every graph reasoning task is insensitive to quantization.

The practical advantage of this framing is falsifiability. Soft-MoE routing can be inspected through slot entropy and expert mass rather than treated as magic capacity. Parameter-Golf claims can be inspected through exact byte accounting, run logs, and validation rules rather than informal model-size estimates. If the model violates relabeling equivariance, the error is measured. If tropical heads collapse to a single global key, active-face entropy shows it. If toric channels memorize denominators, held-out q intervals reveal it. If Hebrew root abstraction fails, held-out root families and paradigm completion expose it. If PolarQuant breaks long-context retrieval, margin-preservation and exact-vs-quantized graph outputs show where.

18 Conclusion

The extended ToricGT framework gives a rigorous path from graph-tokenized Transformers to algebraic, tropical, and morphological graph reasoning. The central mathematical claims are exact finite statements: tokenization and layers are equivariant; tropical ring attention exactly equals full tropical attention; max-plus heads simulate bounded semiring algorithms; finite Weyl pairs and cocycles provide controlled toric features; root-template Hebrew morphology becomes a typed graph problem; Soft-MoE blocks preserve graph equivariance while routing typed computations;

Parameter-Golf compression defines a separate micro-LM deployment path with byte-level constraints; and PolarQuant-style cache compression admits explicit attention perturbation bounds. The resulting research program is small enough for single-GPU experimentation yet structured enough to support serious mathematical diagnostics.

A Additional proof details

A.1 Hybrid universal approximation

Theorem A.1 (Hybrid ToricGT universality). *Let F be a continuous equivariant graph-to-graph map on a bounded compact graph domain. A ToricGT model with at least one ordinary ReLU feed-forward path or softmax attention path and any number of tropical, toric, Hebrew, or PolarQuant-compatible residual paths can approximate F uniformly while preserving equivariance.*

Proof. By graph-to-graph universality, an equivariant TokenGT approximant exists using ordinary Transformer blocks. Add the additional paths as residual maps. Initialize their output projections to zero or arbitrarily small norm, so the augmented model initially equals or approximates the original approximant. Every added path is equivariant by the preceding lemmas when its masks, transports, and quantizers are shared or equivariant. A sum of equivariant maps is equivariant, and the approximation error can be kept below ε by choosing the original approximant error below $\varepsilon/2$ and the residual perturbation below $\varepsilon/2$. $\square$

A.2 Softmax perturbation with quantized keys

Lemma A.2 (Softmax attention perturbation). *Let rows of V and $\hat{V}$ be bounded by R_V in ℓ_∞ , and suppose $\|S - \hat{S}\|_\infty \leq \epsilon_S$ and $\|V - \hat{V}\|_\infty \leq \epsilon_V$. Then a single softmax head satisfies*

$$\left\| \text{softmax}(S)V - \text{softmax}(\hat{S})\hat{V} \right\|_\infty \leq C_L R_V \epsilon_S + \epsilon_V, \quad (\text{A.1})$$

where C_L is a Lipschitz constant for row-wise softmax on L entries.

Proof. Add and subtract $\text{softmax}(\hat{S})V$:

$$\left\| \text{softmax}(S)V - \text{softmax}(\hat{S})\hat{V} \right\|_\infty \leq \left\| (\text{softmax}(S) - \text{softmax}(\hat{S}))V \right\|_\infty + \left\| \text{softmax}(\hat{S})(V - \hat{V}) \right\|_\infty. \quad (\text{A.2})$$

The second term is at most ϵ_V because each softmax row is a probability vector. The first term is bounded by the Lipschitz constant of softmax times ϵ_S times the value bound. This yields the claim. $\square$

B Graph record schema

A recommended JSONL record has fields

`id`, `source`, `task_family`, `nodes`, `edges`, `targets`, `metadata`, `split_cluster`

Each node contains a stable local id, type, symbolic payload, numerical features, source span, and optional uncertainty. Each edge contains source id, target id, type, directionality, numerical features, and optional proof or analyzer metadata. Targets include graph-level labels, node labels,

edge labels, next-state graphs, answer strings, verifier scores, GFlowNet rewards, and visualization payloads.

For Hebrew records, node types include `root`, `radical`, `template`, `binyan`, `feature`, `surface`, `lemma`, `gloss`, `span`, and `hypothesis`. Edge types include `has_radical`, `fills_slot`, `has_feature`, `attested_at`, `same_root`, and `paradigm_member`.

C Recommended hyperparameters

Setting	Value	Notes
Smoke model	2 layers, $d = 64$	CPU-only shape and equivariance tests
8M model	6 layers, $d = 256$	first real training run
15M model	8 layers, $d = 384$	preferred single-4090 target
35M model	12 layers, $d = 512$	checkpointing and careful memory profiling
Attention block size	128–512	tune by sequence length and memory
Tropical heads	25–50% of heads after warmup	avoid early collapse
Torus regularizer	small, delayed	introduce after normal forms are learned
Hebrew loss weight	curriculum-dependent	start with analyzer-supervised tasks
Soft-MoE full model	$E = 4\text{--}8$, $S = 1\text{--}4$	enabled by default in upper layers; monitor expert mass and slot entropy
Soft-MoE Parameter-Golf	$E = 2\text{--}4$, $S = 1\text{--}2$	use shared low-rank adapters and QAT; measure bytes after every change
Parameter-Golf micro-LM	tied embeddings, recurrent 4–6 unique blocks	target under 16,000,000 artifact bytes with train/eval logs
PolarQuant cache	inference/eval first	compare full KV vs compressed cache
Optimizer	AdamW	clipping at 1.0, cosine decay
Precision	bf16	fallback to fp16/fp32 if unsupported
Splits	clustered 80/10/10	no random record-level split

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sanjiv Kumar, and Sashank J. Reddi. Are Transformers universal approximators of sequence-to-sequence functions? *ICLR*, 2020. arXiv:1912.10077.

- [3] Jinwoo Kim, Tien Dat Nguyen, Seonwoo Min, Sungjun Cho, Moontae Lee, Honglak Lee, and Seunghoon Hong. Pure Transformers are powerful graph learners. *NeurIPS*, 2022. arXiv:2207.02505.
- [4] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring Attention with blockwise Transformers for near-infinite context. arXiv:2310.01889, 2023.
- [5] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. Tropical Attention: Neural algorithmic reasoning for combinatorial algorithms. *NeurIPS*, 2025. arXiv:2505.17190.
- [6] Marc A. Rieffel. C^* -algebras associated with irrational rotations. *Pacific Journal of Mathematics*, 93(2):415–429, 1981.
- [7] Marc A. Rieffel. Projective modules over higher-dimensional non-commutative tori. *Canadian Journal of Mathematics*, 40(2):257–338, 1988.
- [8] Mihai Pimsner and Dan Voiculescu. Imbedding the irrational rotation C^* -algebra into an AF-algebra. *Journal of Operator Theory*, 4:201–210, 1980.
- [9] Alain Connes. C^* -algebres et geometrie differentielle. *Comptes Rendus de l’Academie des Sciences*, 290:599–604, 1980.
- [10] Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J. Hu, Mo Tiwari, and Emmanuel Bengio. GFlowNet foundations. *Journal of Machine Learning Research*, 24(210):1–55, 2023.
- [11] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. arXiv:2201.13259, 2022.
- [12] Kanika Madan, Jarrod Rector-Brooks, Maksym Korablyov, Emmanuel Bengio, Moksh Jain, Andrei Cristian Nica, Tom Bosc, Yoshua Bengio, and Nikolay Malkin. Learning GFlowNets from partial episodes for improved convergence and stability. In *ICML*, 2023.
- [13] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, and others. Training compute-optimal large language models. In *NeurIPS*, 2022. arXiv:2203.15556.
- [14] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From Sparse to Soft Mixtures of Experts. *ICLR*, 2024. arXiv:2308.00951.
- [15] OpenAI. Model Craft Challenge: Parameter Golf. GitHub repository <https://github.com/openai/parameter-golf>, accessed May 26, 2026.
- [16] Amelie Schreiber. Parameter Golf fork for ToricGT experimentation. GitHub repository <https://github.com/amelie-iska/parameter-golf>, accessed May 26, 2026.
- [17] Amelie Schreiber. Soft mixture-of-experts fork for ToricGT experimentation. GitHub repository <https://github.com/amelie-iska/soft-mixture-of-experts>, accessed May 26, 2026.
- [18] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV Caches with Polar Transformation. arXiv:2502.02617, 2025.

- [19] Wilhelm Gesenius, edited and enlarged by E. Kautzsch, translated by A. E. Cowley. *Gesenius' Hebrew Grammar*. Oxford University Press, 1910.
- [20] Bruce K. Waltke and M. O'Connor. *An Introduction to Biblical Hebrew Syntax*. Eisenbrauns, 1990.
- [21] Paul Joüon and Takamitsu Muraoka. *A Grammar of Biblical Hebrew*. Gregorian and Biblical Press, revised edition, 2006.
- [22] Kimmo Koskeniemi. Two-level morphology: A general computational model for word-form recognition and production. University of Helsinki, Department of General Linguistics, 1983.
- [23] Kenneth R. Beesley and Lauri Karttunen. *Finite State Morphology*. CSLI Publications, 2003.
- [24] Shlomo Yona and Shuly Wintner. A finite-state morphological grammar of Hebrew. *Natural Language Engineering*, 14(2):173–190, 2008.